

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO BÀI TẬP LỚN
LẬP TRÌNH ỨNG DỤNG MOBILE

Đề tài: RideUp - Ứng dụng đặt xe ghép liên tỉnh

Sinh viên thực hiện: Phạm Quang Huy
Giảng viên hướng dẫn: Nguyễn Hoàng Anh
Nhóm QLDT: 07
Nhóm BTL: 04

Thành viên nhóm	Mã sinh viên
Nguyễn Xuân Hưng	B22DCCN418
Vũ Sỹ Ngọc Hiếu	B22DCCN322
Phạm Quang Huy	B22DCCN394
Khúc Văn Tuấn	B22DCCN754
Nguyễn Vũ Tuấn Khôi	B22DCCN466

DANH SÁCH THÀNH VIÊN VÀ ĐÓNG GÓP

STT	Họ và tên	MSSV	Đóng góp cụ thể
1	Phạm Quang Huy	B22DCCN394	Tài xế: Đăng ký/cập nhật hồ sơ tài xế, tạo chuyến đi, quản lý trạng thái chuyến, xem danh sách & chi tiết chuyến, xem danh sách booking
2	Nguyễn Xuân Hưng	B22DCCN418	Tìm kiếm chuyến , danh sách booking của user, chatbot hỗ trợ, đặt xe
3	Vũ Sỹ Ngọc Hiếu	B22DCCN322	Thanh toán VNPay, Nhắn tin trực tuyến, Thông báo
4	Khúc Văn Tuấn	B22DCCN754	Đăng kí , đăng nhập, Cập nhật thông tin người dùng, Đánh giá chuyên
5	Nguyễn Vũ Tuấn Khôi	B22DCCN466	Báo cáo thống kê, tất cả các chức năng phía role admin (duyet hồ sơ, quản lý user, quản lý thông tin địa lý hành chính)

Bảng 1.1 – Danh sách thành viên và phân công nhiệm vụ

MỤC LỤC

CHƯƠNG 1. MỞ ĐẦU	9
1.1. Giới thiệu ứng dụng	9
1.2. Lý do thực hiện	9
1.3. Concept và mục tiêu.....	9
CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ TỔNG QUAN.....	11
2.1. Kiến trúc tổng quan hệ thống.....	11
2.2. Phân tích yêu cầu chức năng tổng quan.....	12
2.3. Biểu đồ Use Case tổng quan	12
2.4. Yêu cầu phi chức năng	14
CHƯƠNG 3. THIẾT KẾ CHI TIẾT MODULE TÀI XẾ.....	15
3.1. Biểu đồ Use Case chi tiết – Module Tài xế.....	15
3.2. Mô tả chi tiết các Use Case chính	16
3.2.1. UC-D01/D02/D03 – Đăng ký, cập nhật và nộp hồ sơ tài xế	16
3.2.2. UC-D04 – Tạo chuyến đi.....	17
3.2.3. UC-D05/D06/D07 – Cập nhật trạng thái chuyến.....	18
3.2.4. UC-D08/D09 – Xem danh sách và chi tiết chuyến.....	19
3.2.5. UC-D10/D11 – Xem danh sách booking và xác nhận thanh toán	20
3.3. Biểu đồ lớp (Class Diagram) – Module Tài xế.....	20
3.4. Sơ đồ thực thể quan hệ (ER Diagram)	22
3.5. Biểu đồ tuần tự (Sequence Diagram)	24
3.5.1. Luồng đăng ký và nộp hồ sơ tài xế	24
3.5.2. Luồng tạo chuyến đi.....	25
3.5.3. Luồng bắt đầu và hoàn thành chuyến.....	26
3.5.4. Luồng hủy chuyến và hoàn tiền tự động.....	27
3.5.5. Luồng xem booking và xác nhận thanh toán tiền mặt	28
3.6. Thiết kế giao diện và luồng màn hình.....	29
CHƯƠNG 4. KẾT QUẢ THỰC HIỆN	30
4.1. Mô hình triển khai ứng dụng.....	30
4.2. Các bước cài đặt và triển khai	30
4.2.1. Backend (Spring Boot).....	30
4.2.2. Mobile App (React Native / Expo)	31
4.3. Kết quả thực hiện – Các chức năng Module Tài xế.....	31
4.3.1. Chức năng Đăng ký và cập nhật hồ sơ tài xế.....	32

4.3.2. Chức năng Tạo chuyến đi	32
4.3.3. Chức năng Xem danh sách chuyến	34
4.3.4. Chức năng Xem chi tiết chuyến	35
4.3.5. Chức năng Xem danh sách booking	35
4.4. Kết quả thử nghiệm	36
4.5. Dữ liệu thử nghiệm trong hệ thống	38
CHƯƠNG 5. KẾT LUẬN.....	39
5.1. Kết quả đạt được	39
5.2. Hạn chế và điểm cần cải thiện	39
5.3. Hướng phát triển	40
TÀI LIỆU THAM KHẢO.....	41
PHỤ LỤC I . CODE ĐÁP ỨNG CHỨC NĂNG.....	42
1.1. Các lớp Entity và Bảng trong CSDL.....	42
1.1.1. Entity User – Bảng user	42
1.1.2. Entity DriverProfile – Bảng driver_profile.....	43
1.1.3. Entity Vehicle – Bảng vehicle	44
1.1.4. Entity Trip – Bảng trip	44
1.1.5. Entity TripPickupPoint & TripDropoffPoint	45
1.1.6. Entity Booking – Bảng booking.....	46
1.1.7. Entity Payment – Bảng payment.....	47
1.1.8. Entity Ward & Province – Bảng ward, province	47
1.2. Các Controller – Lớp xử lý HTTP Request	48
1.2.1. DriverTripController	48
1.2.2. DriverProfileController	49
1.2.3. FileController	49
1.2.4. LocationController	50
1.3. Các Service – Lớp xử lý nghiệp vụ.....	50
1.3.1. DriverTripService	50
1.3.2. DriverProfileService	52
1.3.3. FileService.....	54
1.3.4. NotificationRealtimePublisher	54
1.4. Các Repository – Lớp truy cập CSDL	54
1.4.1. TripRepository	55
1.4.2. DriverProfileRepository	55

1.4.3. WardRepository	56
1.5. Các DTO (Data Transfer Object).....	56
1.6. Các API Gọi Ngoài (External API).....	57
1.6.1. Supabase Storage REST API	57
1.6.2. VNPay Refund API.....	58
1.6.3. WebSocket STOMP – Thông báo Realtime	58
PHỤ LỤC II . PHẦN CODE – CÁC FILE LIÊN QUAN ĐẾN CÁ NHÂN	
THỰC HIỆN	60
2.1. Backend – Spring Boot (thư mục RideUp/).....	60
2.2. Mobile App – React Native (thư mục RideUpUI/).....	61

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Giải nghĩa
API	Application Programming Interface – Giao diện lập trình ứng dụng
REST	REpresentational State Transfer – Phong cách thiết kế kiến trúc web service
JWT	JSON Web Token – Chuẩn xác thực và truyền thông tin qua token mã hóa
CRUD	Create, Read, Update, Delete – Bốn thao tác cơ bản trên dữ liệu
DB	Database – Cơ sở dữ liệu
DTO	Data Transfer Object – Đối tượng truyền dữ liệu giữa các tầng
SSE	Server-Sent Events – Kỹ thuật server đẩy sự kiện realtime tới client
ORM	Object-Relational Mapping – Ánh xạ đối tượng và bảng quan hệ
UI	User Interface – Giao diện người dùng
UX	User Experience – Trải nghiệm người dùng
CCCD	Căn cước công dân
GPLX	Giấy phép lái xe
VNPay	Vietnam Payment – Cổng thanh toán trực tuyến của Việt Nam
MAD	Mobile Application Development – Lập trình ứng dụng di động

Bảng 1.2 – Danh mục từ viết tắt

DANH MỤC HÌNH VẼ

- Hình 2.1 – Kiến trúc tổng quan hệ thống Ride Up
- Hình 2.2 – Biểu đồ Use Case tổng quan hệ thống
- Hình 3.1 – Biểu đồ Use Case chi tiết – Module Tài xế
- Hình 3.2 – Biểu đồ lớp – Module Tài xế
- Hình 3.3 – Sơ đồ ER – Các bảng liên quan Role Tài xế
- Hình 3.4 – Biểu đồ tuần tự – Đăng ký và nộp hồ sơ tài xế
- Hình 3.5 – Biểu đồ tuần tự – Tạo chuyến đi
- Hình 3.6 – Biểu đồ tuần tự – Cập nhật trạng thái chuyến (Bắt đầu/Hoàn thành)
- Hình 3.7 – Biểu đồ tuần tự – Hủy chuyến và hoàn tiền
- Hình 3.8 – Biểu đồ tuần tự – Xem danh sách booking & xác nhận thanh toán
- Hình 4.1 – Màn hình Đăng ký / Đăng nhập
- Hình 4.2 – Màn hình Hồ sơ tài xế – chưa điền
- Hình 4.3 – Màn hình Hồ sơ tài xế – đã điền đầy đủ
- Hình 4.4 – Màn hình Tạo chuyến đi
- Hình 4.5 – Màn hình Danh sách chuyến (AllTrips)
- Hình 4.6 – Màn hình Chi tiết chuyến đi
- Hình 4.7 – Màn hình Chi tiết chuyến – Danh sách booking
- Hình 4.8 – Màn hình Home tài xế – Thống kê

DANH MỤC BẢNG BIỂU

- Bảng 1.1 – Danh sách thành viên và phân công nhiệm vụ
- Bảng 1.2 – Danh mục từ viết tắt
- Bảng 2.1 – Các thành phần kiến trúc hệ thống
- Bảng 2.2 – Yêu cầu chức năng tổng quan
- Bảng 3.1 – Danh sách Use Case – Module Tài xế
- Bảng 3.2 – Mô tả Use Case Đăng ký/cập nhật hồ sơ tài xế
- Bảng 3.3 – Mô tả Use Case Tạo chuyến đi
- Bảng 3.4 – Mô tả Use Case Cập nhật trạng thái chuyến
- Bảng 3.5 – Mô tả Use Case Xem danh sách và chi tiết chuyến
- Bảng 3.6 – Mô tả Use Case Xem danh sách booking
- Bảng 4.1 – Mô hình triển khai ứng dụng
- Bảng 4.2 – Kết quả kiểm thử các chức năng

CHƯƠNG 1. MỞ ĐẦU

1.1. Giới thiệu ứng dụng

Ride Up là ứng dụng di động phục vụ nhu cầu đặt xe liên tỉnh theo mô hình ghép chỗ (ride-sharing). Ứng dụng cho phép tài xế có phương tiện cá nhân đăng ký và tạo các chuyến đi liên tỉnh, đồng thời cho phép hành khách tìm kiếm và đặt chỗ trực tiếp trên ứng dụng với mức giá cố định và linh hoạt về lịch trình.

Ứng dụng được phát triển trên nền tảng **React Native (Expo)** cho phần giao diện di động và **Spring Boot 3.x** cho phần backend, sử dụng cơ sở dữ liệu **MySQL** (thông qua Supabase), **MongoDB** cho dữ liệu chat, **Redis** cho cache và **VNPay** cho thanh toán trực tuyến.

Hệ thống hỗ trợ ba nhóm người dùng: **Tài xế (Driver)** – tạo và quản lý chuyến đi; **Hành khách (Customer)** – tìm kiếm và đặt chỗ; **Quản trị viên (Admin)** – duyệt hồ sơ tài xế và quản lý hệ thống.

1.2. Lý do thực hiện

Nhu cầu di chuyển liên tỉnh tại Việt Nam rất lớn. Các phương tiện truyền thống như xe khách, xe giường nằm và tàu hỏa mặc dù phổ biến nhưng còn nhiều hạn chế: giá vé không linh hoạt, điểm đón trả cố định, và lịch trình không thể điều chỉnh. Trong khi đó, hàng nghìn tài xế có nhu cầu di chuyển liên tỉnh mỗi ngày nhưng chưa có nền tảng để kết nối và chia sẻ chỗ trống.

Ứng dụng Ride Up được thực hiện nhằm giải quyết những vấn đề đó, tạo ra một nền tảng kết nối tài xế và hành khách theo hướng chia sẻ chỗ đi liên tỉnh, giúp giảm chi phí cho cả hai bên và tận dụng nguồn lực phương tiện hiện có.

1.3. Concept và mục tiêu

Ứng dụng được thiết kế theo mô hình **marketplace hai chiều**: tài xế tự đăng chuyến đi kèm giá cố định trên từng ghế, hành khách tìm kiếm và đặt chỗ trên các chuyến đó. Mục tiêu cụ thể:

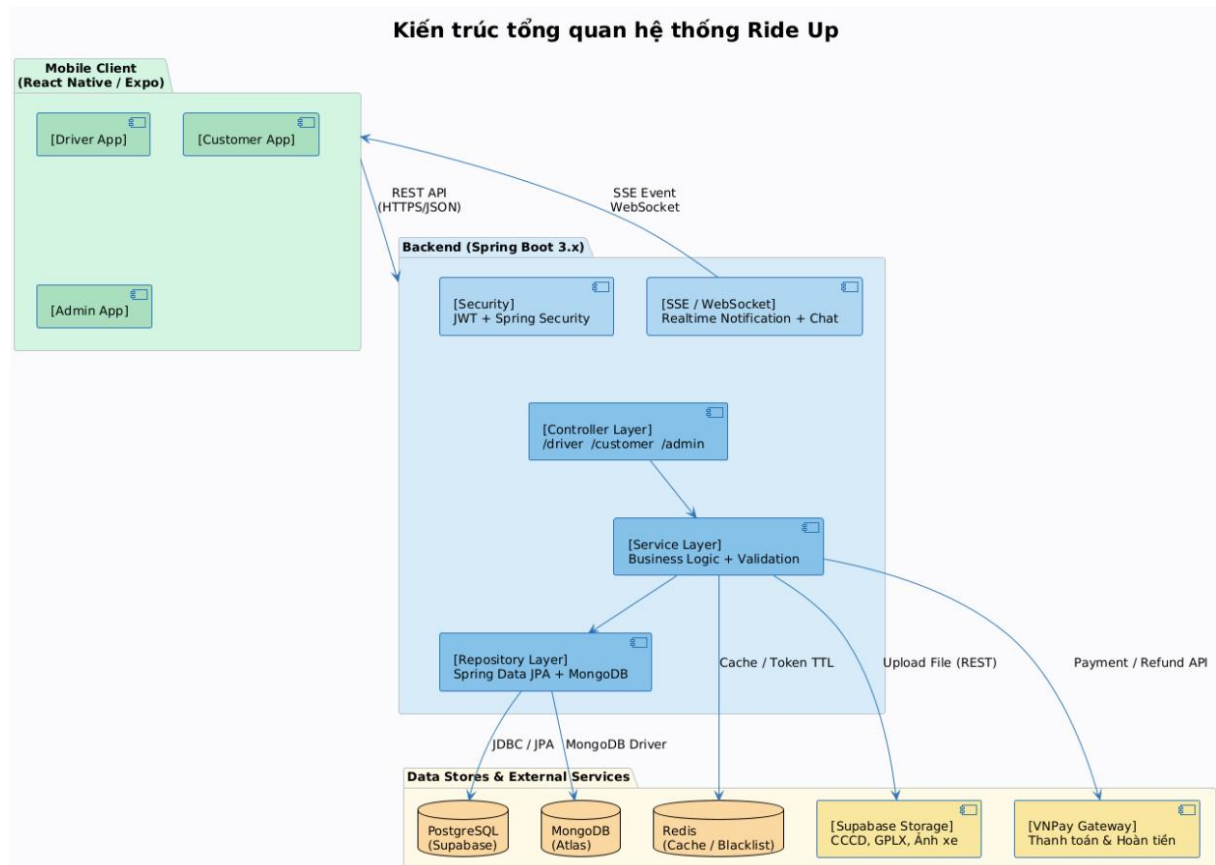
- Xây dựng nền tảng mobile dễ sử dụng, chạy được trên cả Android và iOS

- Xác minh tài xế qua quy trình duyệt hồ sơ (CCCD, GPLX, thông tin xe) bởi Admin
- Hỗ trợ thanh toán online qua VNPay và tiền mặt với hoàn tiền tự động khi hủy chuyến
- Cung cấp thông báo và nhắn tin realtime giữa tài xế và hành khách
- Đảm bảo bảo mật qua xác thực JWT và phân quyền theo role

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ TỔNG QUAN

2.1. Kiến trúc tổng quan hệ thống

Hệ thống Ride Up được xây dựng theo mô hình kiến trúc **Client – Server** nhiều tầng, trong đó phần giao diện di động và phần backend hoàn toàn tách biệt và giao tiếp qua REST API. Hình 2.1 mô tả tổng quan kiến trúc hệ thống.



Hình 2.1 – Kiến trúc tổng quan hệ thống Ride Up

Mô tả các thành phần:

Thành phần	Công nghệ	Mô tả vai trò
Mobile Client	React Native / Expo	Giao diện người dùng cho cả 3 role (Driver, Customer, Admin). Gọi REST API, kết nối SSE realtime và WebSocket chat.
Backend API	Spring Boot 3.x	Xử lý toàn bộ logic nghiệp vụ, phân quyền JWT theo role, expose các REST endpoint, xử lý thanh toán VNPAY, publish sự kiện SSE.
MySQL	Supabase (cloud)	Lưu trữ dữ liệu quan hệ: User, DriverProfile, Vehicle, Trip, TripPickupPoint, TripDropoffPoint,

		Booking, Payment, BookingReview, Ward, Province.
MongoDB	MongoDB Atlas	Lưu trữ dữ liệu chat: ChatThreadDocument (luồng hội thoại) và ChatMessageDocument (tin nhắn).
Redis	Upstash / Railway	Cache API response để giảm tải DB, lưu trữ danh sách refresh token bị vô hiệu hóa, quản lý session realtime.
Supabase Storage	Supabase (cloud)	Lưu trữ file tải lên: ảnh CCCD (mặt trước/sau), ảnh GPLX, avatar, ảnh xe, đăng ký xe, bảo hiểm.
VNPay Gateway	VNPay Sandbox/Live	Xử lý thanh toán trực tuyến và hoàn tiền (refund) tự động khi tài xế hoặc khách hủy chuyến.
SSE / WebSocket	Spring (built-in)	SSE đẩy thông báo realtime đến client; WebSocket cho tính năng chat realtime hai chiều.

Bảng 2.1 – Các thành phần kiến trúc hệ thống

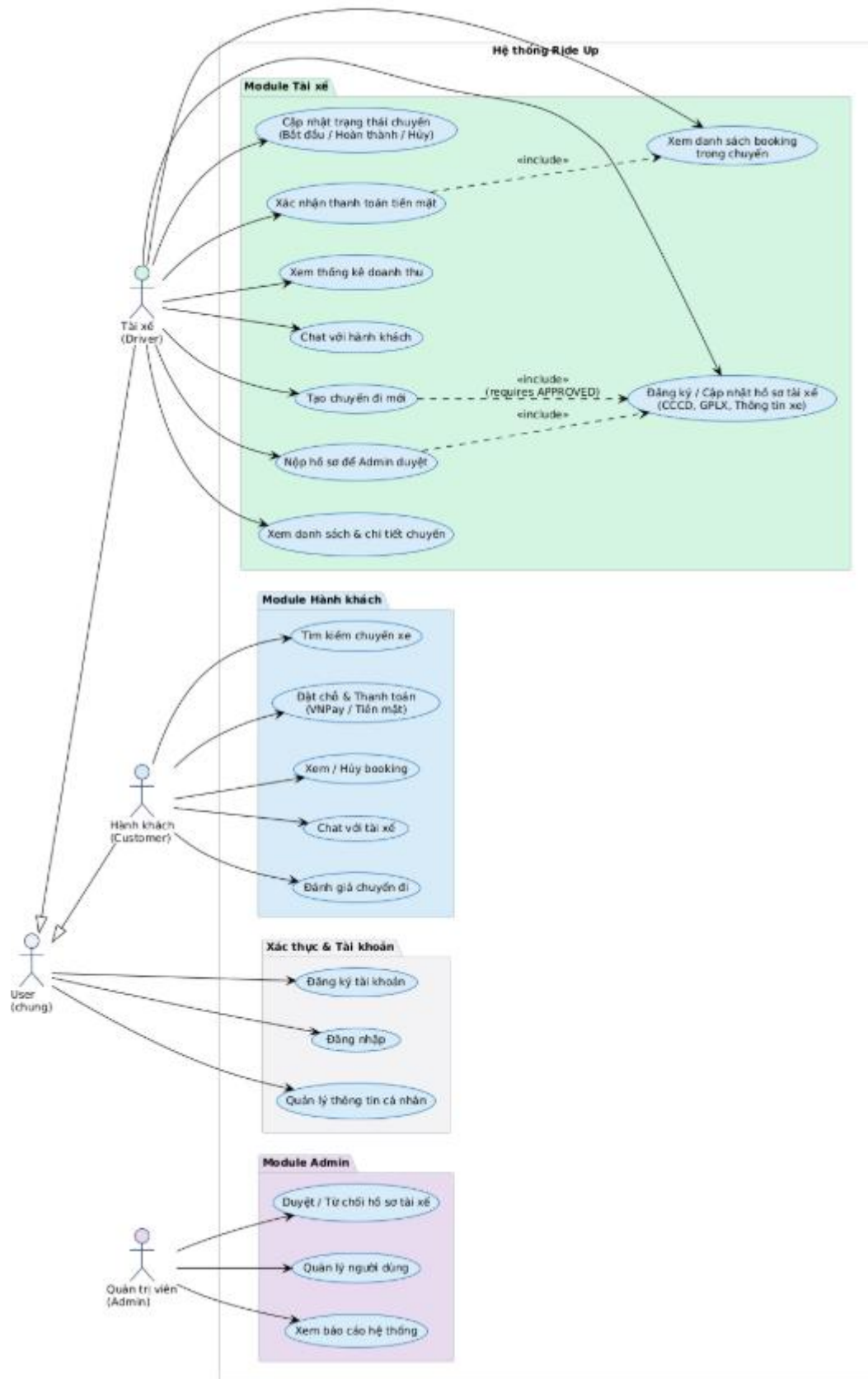
2.2. Phân tích yêu cầu chức năng tổng quan

Hệ thống Ride Up có ba actor chính với các nhóm chức năng tương ứng:

Actor	Nhóm chức năng
Tài xế (Driver)	Đăng ký/cập nhật hồ sơ; tạo chuyến đi; quản lý chuyến (bắt đầu, hoàn thành, hủy); xem danh sách & chi tiết chuyến; xem danh sách booking; xác nhận thanh toán tiền mặt; thông kê doanh thu; chat realtime.
Hành khách (Customer)	Tìm kiếm chuyến xe; xem chi tiết chuyến; đặt chỗ và thanh toán; xem/hủy booking; chat với tài xế; đánh giá chuyến.
Quản trị viên (Admin)	Xem danh sách hồ sơ tài xế chờ duyệt; duyệt/từ chối hồ sơ; quản lý người dùng; xem báo cáo thống kê hệ thống.

Bảng 2.2 – Yêu cầu chức năng tổng quan theo actor

2.3. Biểu đồ Use Case tổng quan



Hình 2.2 – Biểu đồ Use Case tổng quan hệ thống

Biểu đồ Use Case tổng quan thể hiện mối quan hệ giữa ba actor chính (Driver, Customer, Admin) và các nhóm chức năng tương ứng. Actor Driver và

Customer kế thừa chức năng từ actor **User** (đăng nhập, đăng ký, quản lý tài khoản). Admin có quyền truy cập vào các chức năng quản lý riêng biệt không chồng chéo với Driver hay Customer.

2.4. Yêu cầu phi chức năng

- Hiệu năng: API phải trả về response dưới 500ms với hỗ trợ cache Redis; hệ thống đảm bảo tối thiểu 100 người dùng đồng thời
- Bảo mật: Xác thực bằng JWT (access token + refresh token), phân quyền role-based (DRIVER/CUSTOMER/ADMIN); mật khẩu mã hóa BCrypt
- Tính khả dụng: Ứng dụng mobile chạy được trên cả Android (API 26+) và iOS (12+); backend triển khai trên cloud với uptime cao
- Khả năng mở rộng: Kiến trúc layered (Controller – Service – Repository) dễ bảo trì và mở rộng; MongoDB Atlas và MySQL Supabase có khả năng scale tốt
- Trải nghiệm người dùng: Giao diện phản hồi nhanh với skeleton loading; thông báo realtime tức thì qua Websocket Broker

3.1. Biểu đồ Use Case chi tiết – Module Tài xế



Trang 15/62

Mã UC	Tên Use Case	Mô tả ngắn
UC-D01	Đăng ký hồ sơ tài xế	Tài xế điền thông tin cá nhân, CCCD, GPLX và thông tin phương tiện
UC-D02	Cập nhật hồ sơ tài xế	Sửa đổi các thông tin đã điền trước khi nộp hoặc sau khi bị từ chối
UC-D03	Nộp hồ sơ để duyệt	Gửi hồ sơ đến Admin để xem xét và phê duyệt; hồ sơ bị khóa khi đang chờ
UC-D04	Tạo chuyến đi	Nhập tỉnh đi/đến, điểm đón/trả theo xã/phường, giá vé, số ghế, ngày giờ khởi hành
UC-D05	Bắt đầu chuyến	Chuyển trạng thái chuyến sang IN_PROGRESS; ghi lại thời gian thực tế xuất phát
UC-D06	Hoàn thành chuyến	Chuyển trạng thái COMPLETED; tự động hoàn tất tất cả booking đang active
UC-D07	Hủy chuyến	Hủy chuyến trước khi khởi hành; tự động hủy booking và hoàn tiền VNPay
UC-D08	Xem danh sách chuyến	Xem tất cả chuyến đã tạo với bộ lọc theo trạng thái
UC-D09	Xem chi tiết chuyến	Xem thông tin đầy đủ: điểm đón/trả, thống kê ghế, danh sách booking
UC-D10	Xem danh sách booking	Xem chi tiết từng booking trong chuyến: hành khách, điểm đón/trả, thanh toán
UC-D11	Xác nhận tiền mặt	Xác nhận đã thu tiền mặt từ hành khách cho booking có phương thức cash

Bảng 3.1 – Danh sách Use Case Module Tài xế

3.2. Mô tả chi tiết các Use Case chính

3.2.1. UC-D01/D02/D03 – Đăng ký, cập nhật và nộp hồ sơ tài xế

Thuộc tính	Nội dung
Mã Use Case	UC-D01 / UC-D02 / UC-D03
Tên	Đăng ký, cập nhật và nộp hồ sơ tài xế
Actor	Tài xế (Driver)
Điều kiện tiên quyết	Người dùng đã đăng nhập vào hệ thống
Luồng chính	1. Tài xế truy cập màn hình "Hồ sơ tài xế"

	2. Hệ thống gọi GET /driver/profile để lấy thông tin hiện tại (hoặc tạo mới nếu chưa có) 3. Tài xế điền các trường: họ tên, ngày sinh, giới tính, số CCCD, ảnh CCCD mặt trước/sau, số GPLX, ngày hết hạn GPLX, ảnh GPLX 4. Tài xế điền thông tin xe: biển số, hãng xe, dòng xe, năm sản xuất, màu xe, số chỗ ngồi, loại xe, ảnh xe, ảnh đăng ký xe, ảnh bảo hiểm 5. Hệ thống cho phép tải ảnh lên Supabase Storage qua PUT /driver/profile 6. Tài xế bấm "Nộp hồ sơ" → hệ thống gọi POST /driver/profile/submit 7. Backend validate: bắt buộc có CCCD, GPLX, biển số, hãng xe, dòng xe 8. Hồ sơ chuyển sang trạng thái PENDING; trường submitted = true; hồ sơ bị khóa 9. Admin nhận thông báo và duyệt hồ sơ
Luồng thay thế	A1: Thiếu thông tin bắt buộc → hiển thị lỗi validation, không cho submit A2: Hồ sơ bị Admin từ chối → submitted = false; tài xế có thể chỉnh sửa và nộp lại A3: Tài xế sửa hồ sơ sau khi được duyệt → trạng thái tự động chuyển về PENDING, yêu cầu duyệt lại
Hậu điều kiện	Hồ sơ tài xế ở trạng thái PENDING, chờ Admin duyệt. Sau khi duyệt → APPROVED, tài xế có thể tạo chuyến đi.
API liên quan	GET /driver/profile, PUT /driver/profile, POST /driver/profile/submit, POST /files/upload

Bảng 3.2 – Mô tả Use Case UC-D01/D02/D03

3.2.2. UC-D04 – Tạo chuyến đi

Thuộc tính	Nội dung
Mã Use Case	UC-D04
Tên	Tạo chuyến đi mới
Actor	Tài xế (Driver)
Điều kiện tiên quyết	Tài xế đã đăng nhập và hồ sơ có trạng thái APPROVED
Luồng chính	1. Tài xế bấm "Tạo chuyến" trên màn hình Home 2. Hệ thống kiểm tra hồ sơ APPROVED (nếu chưa → chuyển sang trạng hồ sơ) 3. Màn hình CreateTripScreen hiển thị form nhập liệu 4. Tài xế chọn tỉnh/thành xuất phát và đến từ danh sách tỉnh thành

	5. Tài xế chọn các khu vực đón (pickup clusters) trong tỉnh xuất phát 6. Tài xế chọn các khu vực trả (dropoff clusters) trong tỉnh đến 7. Tài xế nhập giá vé (đồng/ghế), chọn số ghế (1–8) bằng stepper 8. Tài xế chọn ngày khởi hành qua calendar picker, chọn giờ qua time slot picker 9. Màn hình hiển thị preview "Doanh thu dự kiến" = giá $\times$ số ghế 10. Tài xế tùy chọn thêm ghi chú 11. Bấm "Tạo chuyến" $\rightarrow$ gọi POST /driver/trips 12. Backend tạo Trip với TripPickupPoints và TripDropoffPoints từ ward IDs 13. Hệ thống gửi thông báo realtime SSE cho tài xế 14. Màn hình hiển thị overlay thành công, reset form, chuyển về Home
Luồng thay thế	A1: Chưa chọn đủ tỉnh/xã/giá/ngày $\rightarrow$ alert "Thiếu thông tin" A2: Giá vé $< 1.000\text{đ}$ $\rightarrow$ alert "Giá không hợp lệ" A3: Ngày khởi hành trong quá khứ $\rightarrow$ disabled trên calendar, không cho chọn A4: Hồ sơ chưa được duyệt $\rightarrow$ redirect sang DriverProfileScreen
Hậu điều kiện	Trip được lưu vào DB với status = OPEN, tài xế nhận được thông báo xác nhận, chuyến xuất hiện trong danh sách
API liên quan	POST /driver/trips, GET /locations/provinces, GET /locations/wards

Bảng 3.3 – Mô tả Use Case UC-D04: Tạo chuyến đi

3.2.3. UC-D05/D06/D07 – Cập nhật trạng thái chuyến

Thuộc tính	Nội dung
Mã Use Case	UC-D05 / UC-D06 / UC-D07
Tên	Bắt đầu / Hoàn thành / Hủy chuyến
Actor	Tài xế (Driver)
Điều kiện tiên quyết	Chuyến đi tồn tại và thuộc sở hữu của tài xế đang đăng nhập
Bắt đầu chuyến	1. Tài xế bấm "Bắt đầu" trên chi tiết chuyến 2. Hệ thống kiểm tra status $\in \{\text{OPEN}, \text{FULL}\}$ 3. Kiểm tra departureTime $\leq$ now (không cho bắt đầu sớm hơn giờ lên lịch) 4. Gọi PUT /driver/trips/{id}/start 5. Trip.status $\rightarrow$ IN_PROGRESS, actualDepartureTime = now 6. Thông báo SSE gửi đến tài xế

Hoàn thành chuyến	1. Tài xế bấm "Hoàn thành" khi đã đến nơi 2. Hệ thống kiểm tra không còn booking tiền mặt chưa xác nhận (nếu còn → alert lỗi) 3. Gọi PUT /driver/trips/{id}/complete 4. Trip.status → COMPLETED; tất cả booking CONFIRMED/PENDING → COMPLETED 5. Chat threads liên quan đóng lại 6. Thông báo SSE gửi đến tài xế và tất cả hành khách trong chuyến
Hủy chuyến	1. Tài xế bấm "Hủy chuyến" (chỉ khả dụng với status OPEN/FULL) 2. Hệ thống xác nhận lý do hủy (tùy chọn) 3. Gọi PUT /driver/trips/{id}/cancel 4. Trip.status → CANCELLED 5. Tất cả booking PENDING/CONFIRMED → CANCELLED_BY_DRIVER 6. Booking thanh toán VNPAY → tự động refund về tài khoản khách 7. Chat threads đóng; thông báo SSE đến tài xế và hành khách
Hậu điều kiện	Trạng thái trip được cập nhật; các booking và payment được cập nhật tương ứng; tất cả bên liên quan nhận thông báo
API liên quan	PUT /driver/trips/{id}/start, PUT /driver/trips/{id}/complete, PUT /driver/trips/{id}/cancel

Bảng 3.4 – Mô tả Use Case UC-D05/D06/D07: Cập nhật trạng thái chuyến

3.2.4. UC-D08/D09 – Xem danh sách và chi tiết chuyến

Thuộc tính	Nội dung
Mã Use Case	UC-D08 / UC-D09
Tên	Xem danh sách và chi tiết chuyến đi
Luồng chính (danh sách)	1. Tài xế truy cập tab "Chuyến xe" (AllTripsScreen) 2. Hệ thống gọi GET /driver/trips → trả về list DriverTripResponse 3. Hiển thị danh sách có filter theo tab: Tất cả / Đã lên lịch / Đang chạy / Hoàn thành / Đã hủy 4. Mỗi card hiển thị: tuyến (tỉnh A → tỉnh B), ngày giờ, trạng thái, số ghế đã đặt/tổng 5. Có thanh progress bar thể hiện tỷ lệ ghế đã đặt
Luồng chính (chi tiết)	1. Tài xế bấm vào một chuyến → TripDetailScreen 2. Hệ thống gọi GET /driver/trips/{id} → DriverTripDetailResponse 3. Hiển thị: tuyến đường, ngày giờ lên lịch, thời gian thực tế, tổng/đặt/còn trống ghế, giá vé, doanh thu dự kiến

	4. Danh sách điểm đón theo thứ tự (sortOrder), điểm trả theo thứ tự 5. Các nút hành động: Bắt đầu / Hoàn thành / Hủy (tùy theo status) 6. Danh sách booking với thông tin từng hành khách
API liên quan	GET /driver/trips, GET /driver/trips/{id}

Bảng 3.5 – Mô tả Use Case UC-D08/D09: Xem danh sách và chi tiết chuyến

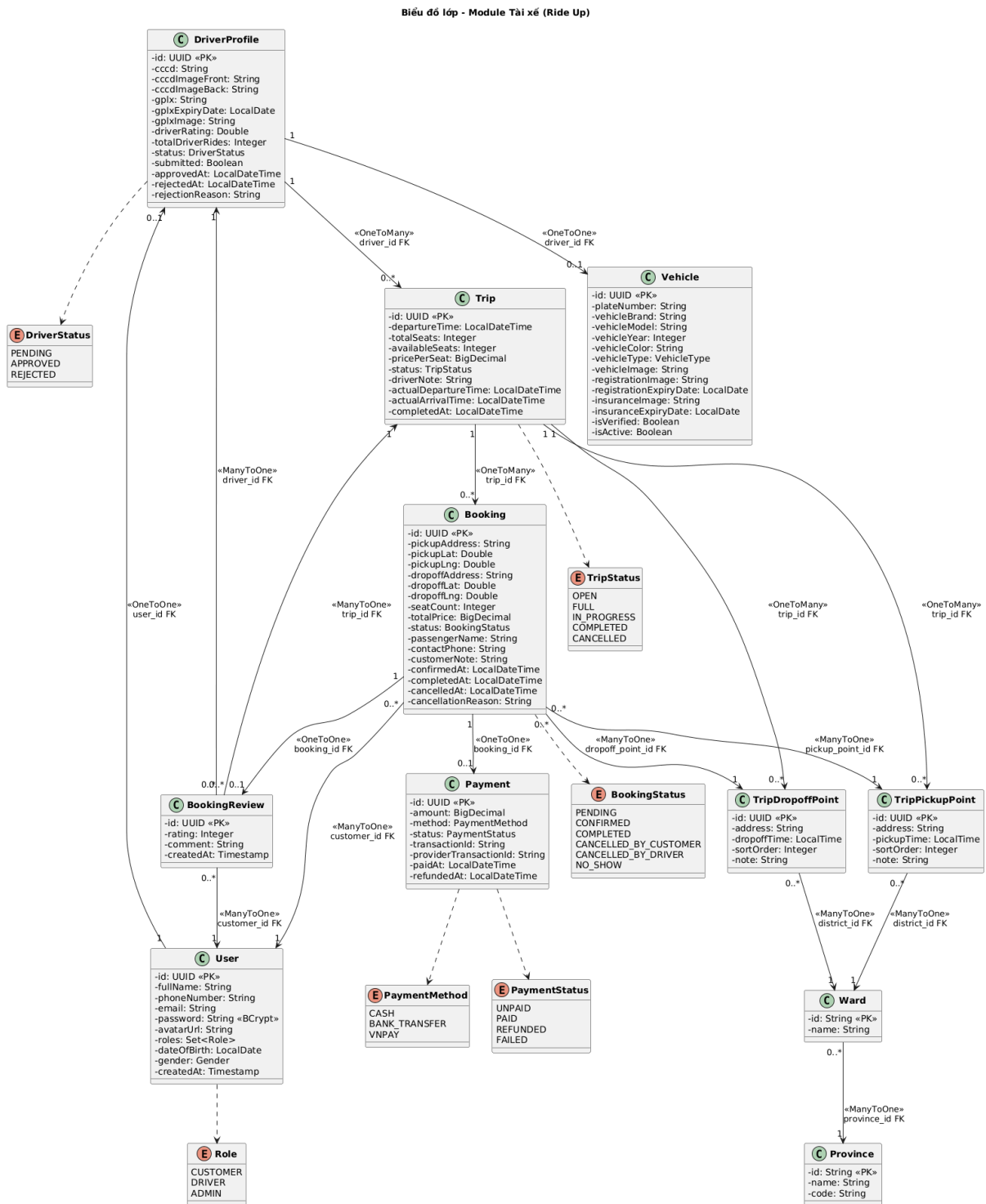
3.2.5. UC-D10/D11 – Xem danh sách booking và xác nhận thanh toán

Thuộc tính	Nội dung
Mã Use Case	UC-D10 / UC-D11
Tên	Xem danh sách booking và xác nhận thanh toán tiền mặt
Luồng chính	1. Trong màn hình chi tiết chuyến, tài xế xem danh sách booking 2. Mỗi booking hiển thị: tên hành khách, SĐT, điểm đón cụ thể, điểm trả cụ thể, số ghế, tổng tiền, phương thức thanh toán, trạng thái thanh toán 3. Với booking có payment.method = CASH và payment.status = UNPAID: hiển thị nút "Xác nhận đã nhận tiền" 4. Tài xế bấm xác nhận → gọi PUT /driver/trips/{id}/bookings/{bookingId}/confirm-cash-payment 5. Backend: payment.status → PAID, payment.paidAt = now 6. Hệ thống gửi thông báo SSE đến hành khách tương ứng
Điều kiện xác nhận	Chỉ xác nhận được booking có payment.method = CASH và payment.status = UNPAID. Không thể xác nhận VNPay (tự động).
API liên quan	GET /driver/trips/{id} (booking list có trong response), PUT /driver/trips/{id}/bookings/{bookingId}/confirm-cash-payment

Bảng 3.6 – Mô tả Use Case UC-D10/D11

3.3. Biểu đồ lớp (Class Diagram) – Module Tài xế

Biểu đồ lớp mô tả các entity chính trong module tài xế và mối quan hệ giữa chúng. Sơ đồ text dưới đây phản ánh chính xác cấu trúc JPA entity trong source code:



Hình 3.2 – Biểu đồ lớp Module Tài xế

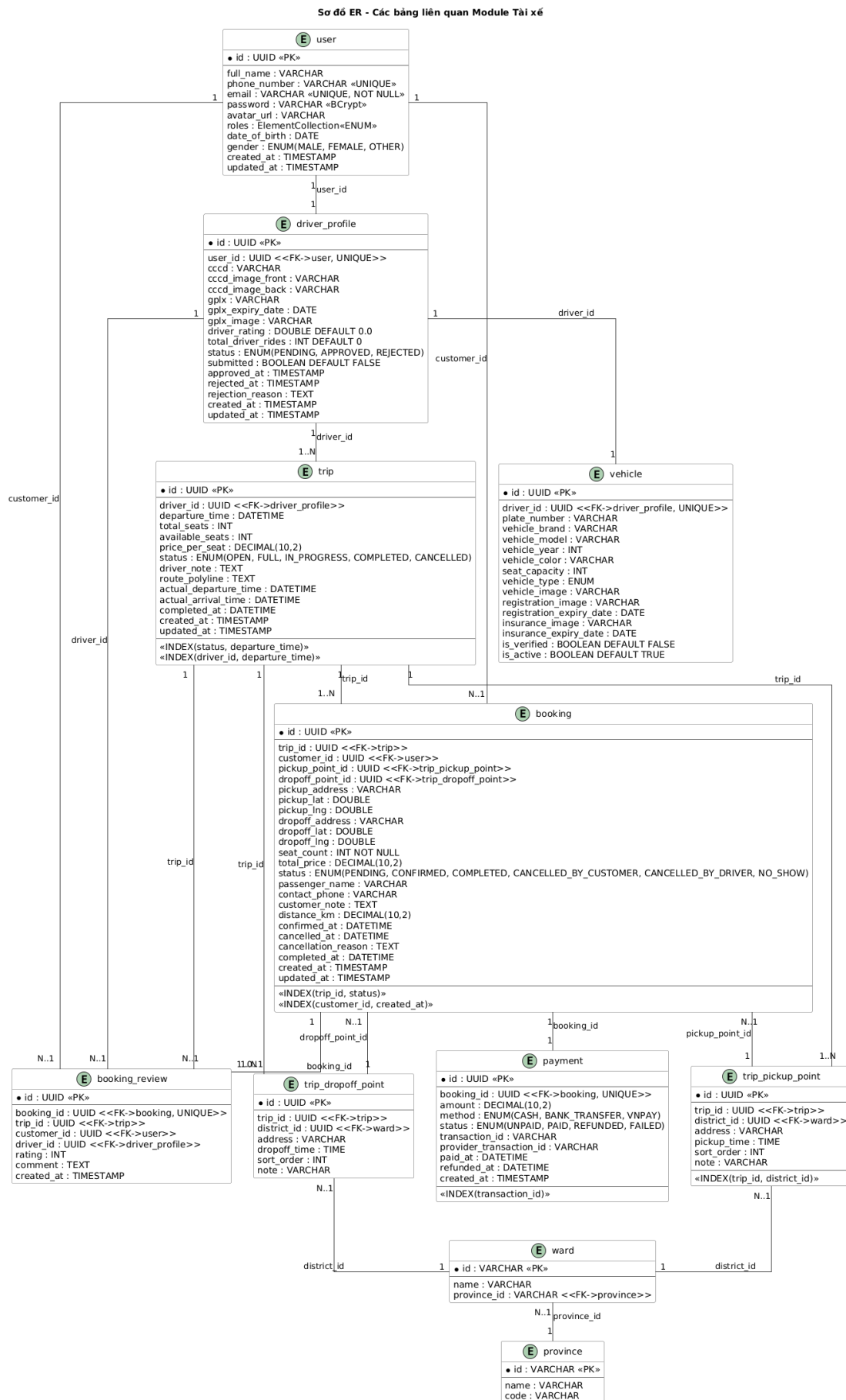
Mô tả các entity chính:

Entity	Vai trò	Thuộc tính quan trọng
User	Người dùng hệ thống – dùng chung cho cả Driver, Customer, Admin	id, fullName, phoneNumber, email, password, roles, avatarUrl

DriverProfile	Hồ sơ tài xế – liên kết 1:1 với User, chứa thông tin xác minh	cccd, gplx, gplxExpiryDate, status, driverRating, submitted
Vehicle	Phương tiện của tài xế – liên kết 1:1 với DriverProfile	plateNumber, vehicleBrand, vehicleModel, seatCapacity, vehicleType, isVerified
Trip	Chuyến đi – do tài xế tạo, liên kết nhiều booking	departureTime, totalSeats, availableSeats, pricePerSeat, status, actualDepartureTime
TripPickupPoint	Điểm đón trong chuyến – liên kết với Ward (xã/phường)	ward, address, pickupTime, sortOrder, note
TripDropoffPoint	Điểm trả trong chuyến – liên kết với Ward (xã/phường)	ward, address, dropoffTime, sortOrder, note
Booking	Đặt chỗ của hành khách trên chuyến – liên kết với Trip, User, Payment	seatCount, totalPrice, status, pickupAddress, dropoffAddress, contactPhone, passengerName
Payment	Thanh toán cho một booking – liên kết 1:1 với Booking	amount, method, status, transactionId, providerTransactionId, paidAt

3.4. Sơ đồ thực thể quan hệ (ER Diagram)

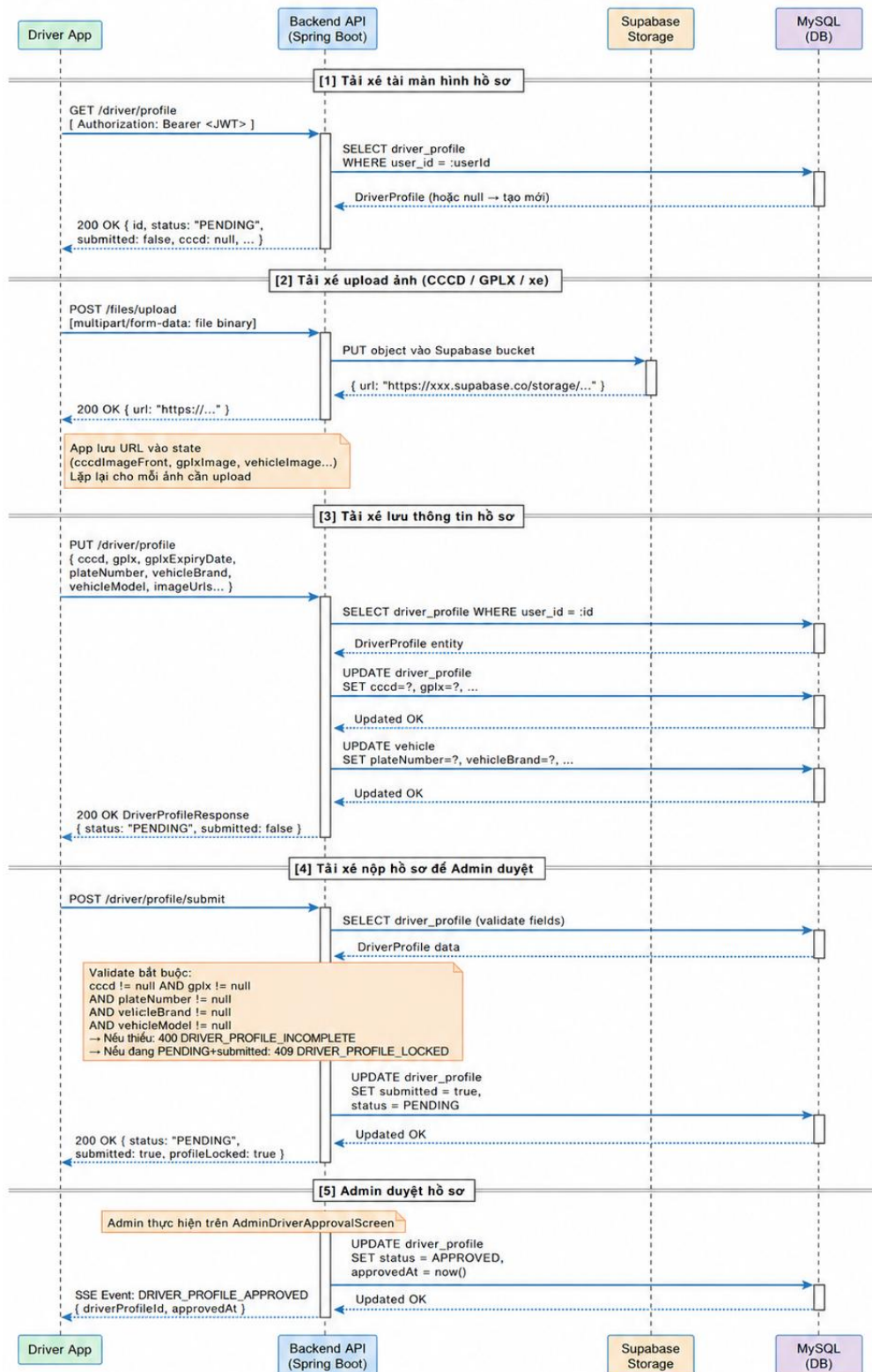
Sơ đồ ER mô tả cấu trúc các bảng trong cơ sở dữ liệu MySQL liên quan đến module Tài xế. Toàn bộ các bảng sử dụng UUID làm khóa chính và có các cột audit (created_at, updated_at).



Hình 3.3 – Sơ đồ ER các bảng liên quan Module Tài xế

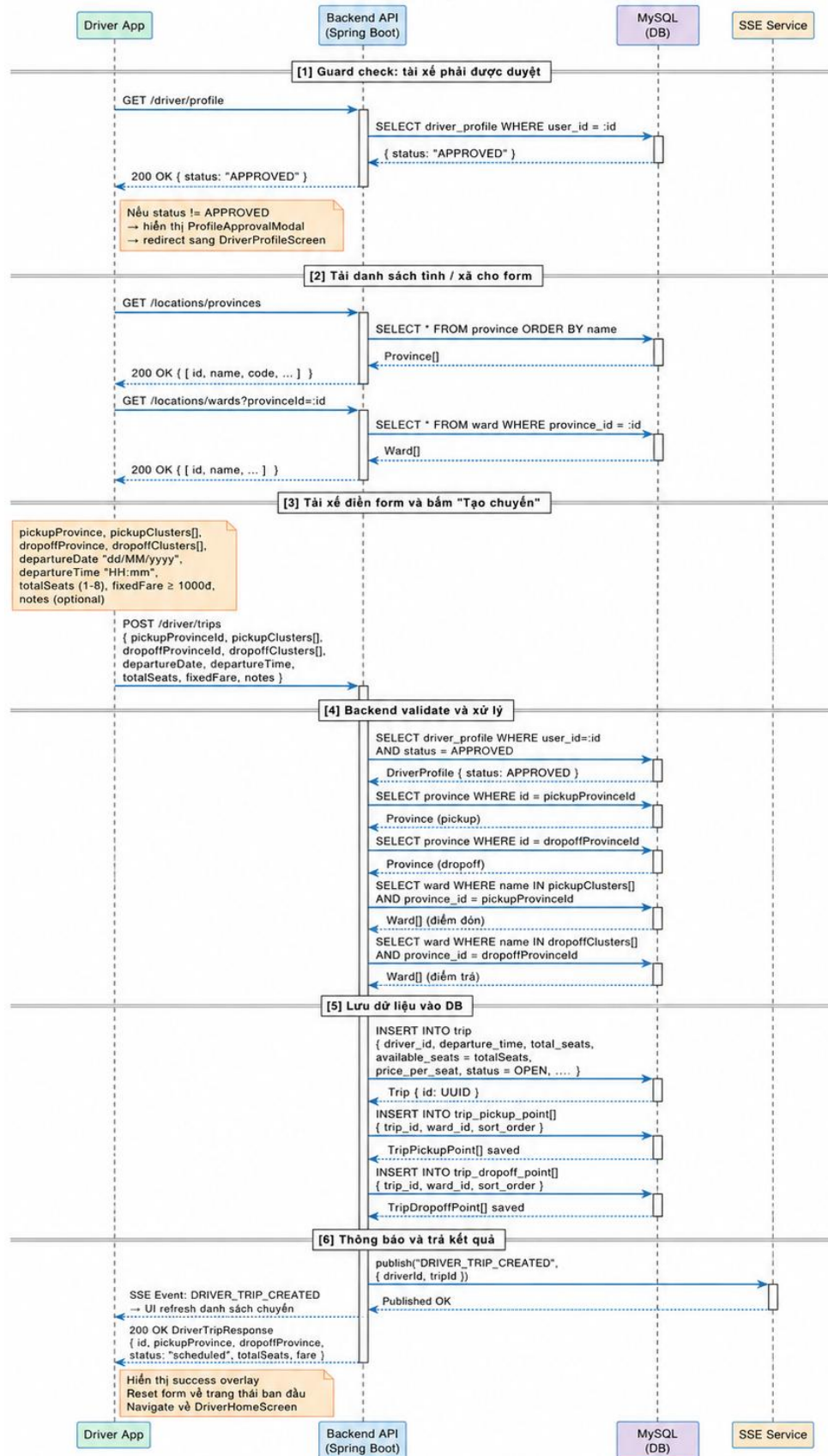
3.5. Biểu đồ tuần tự (Sequence Diagram)

3.5.1. Luồng đăng ký và nộp hồ sơ tài xế



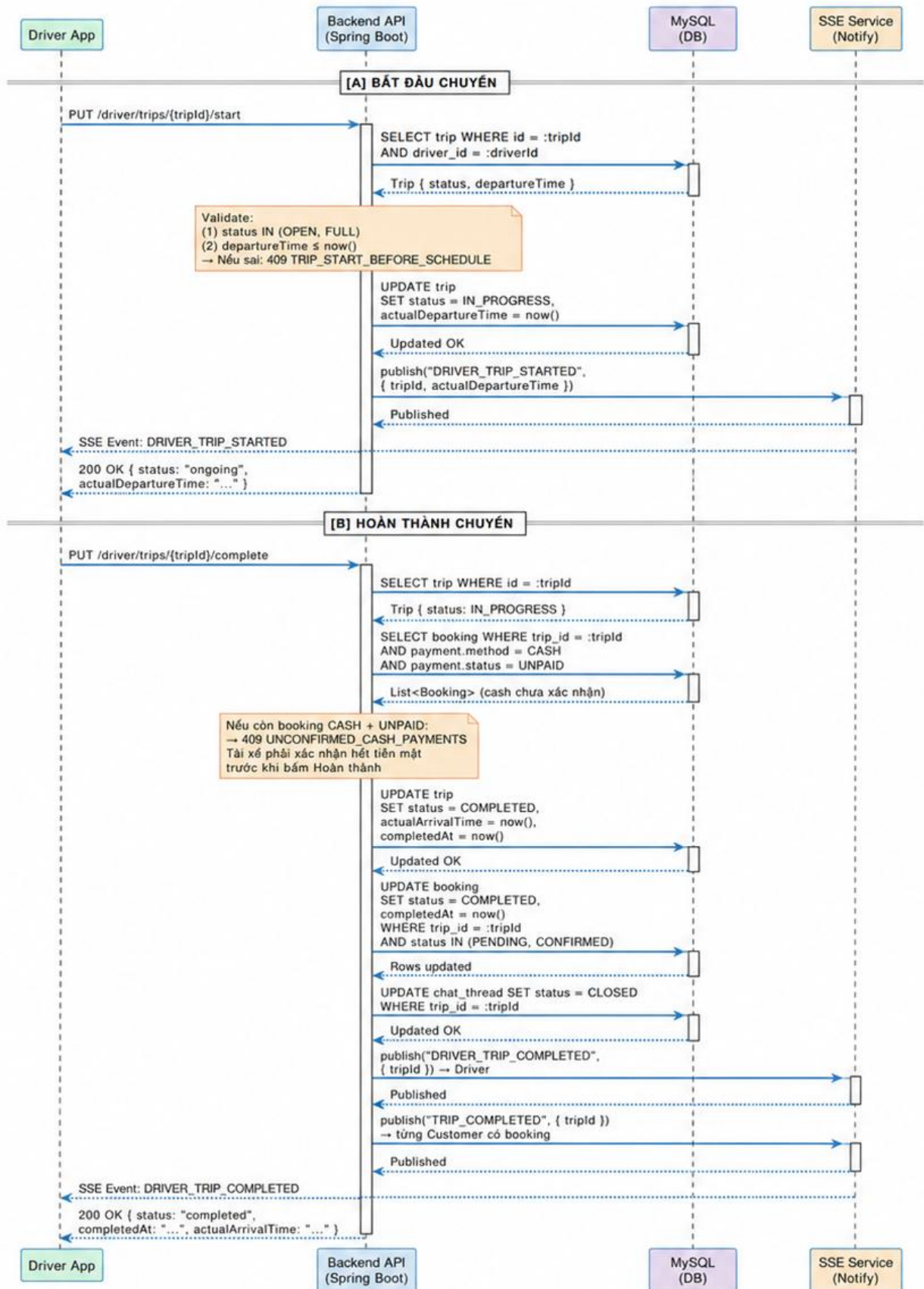
Hình 3.4 – Biểu đồ tuần tự: Đăng ký và nộp hồ sơ tài xế

3.5.2. Luồng tạo chuyến đi



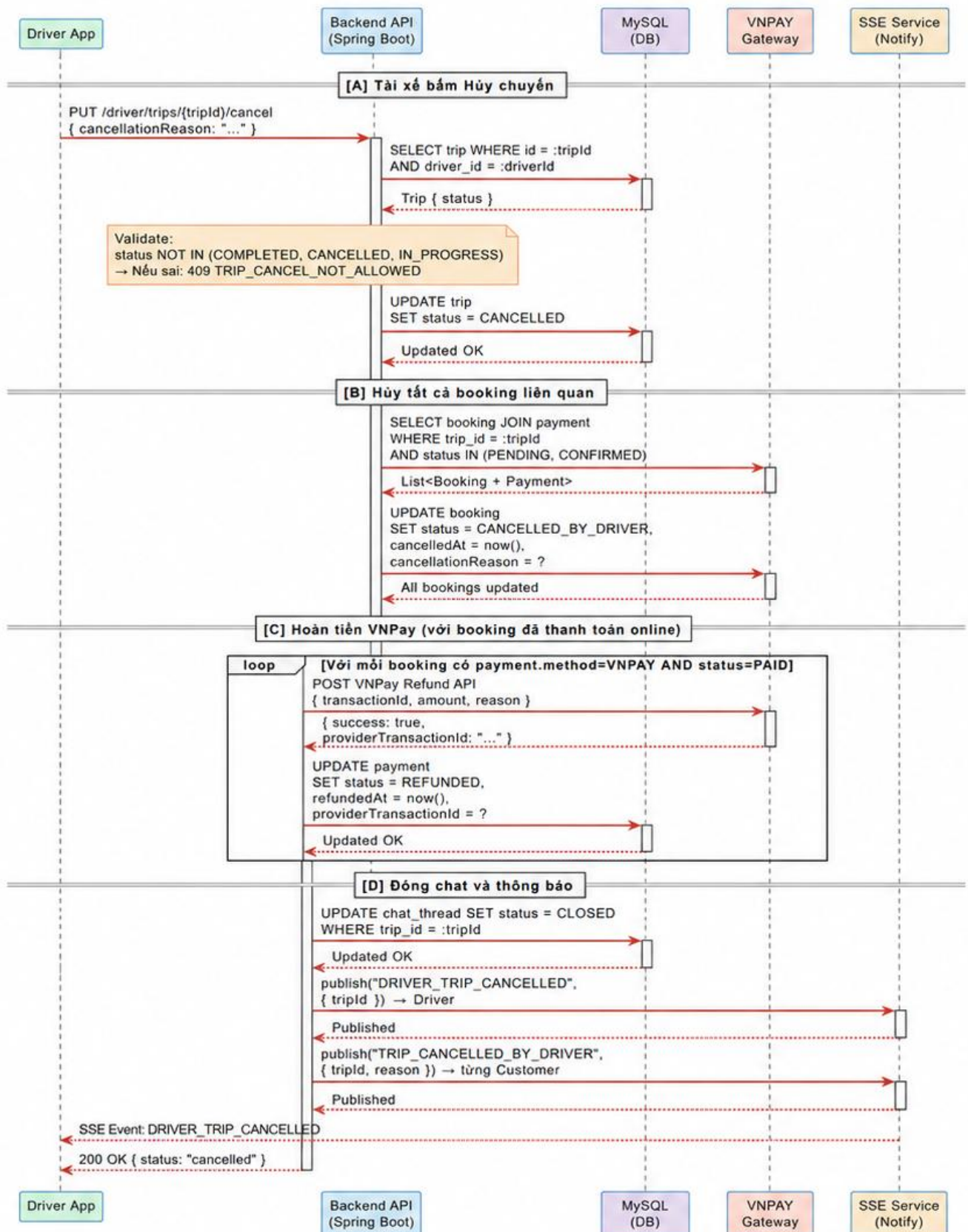
Hình 3.5 – Biểu đồ tuần tự: Tạo chuyến đi

3.5.3. Luồng bắt đầu và hoàn thành chuyến



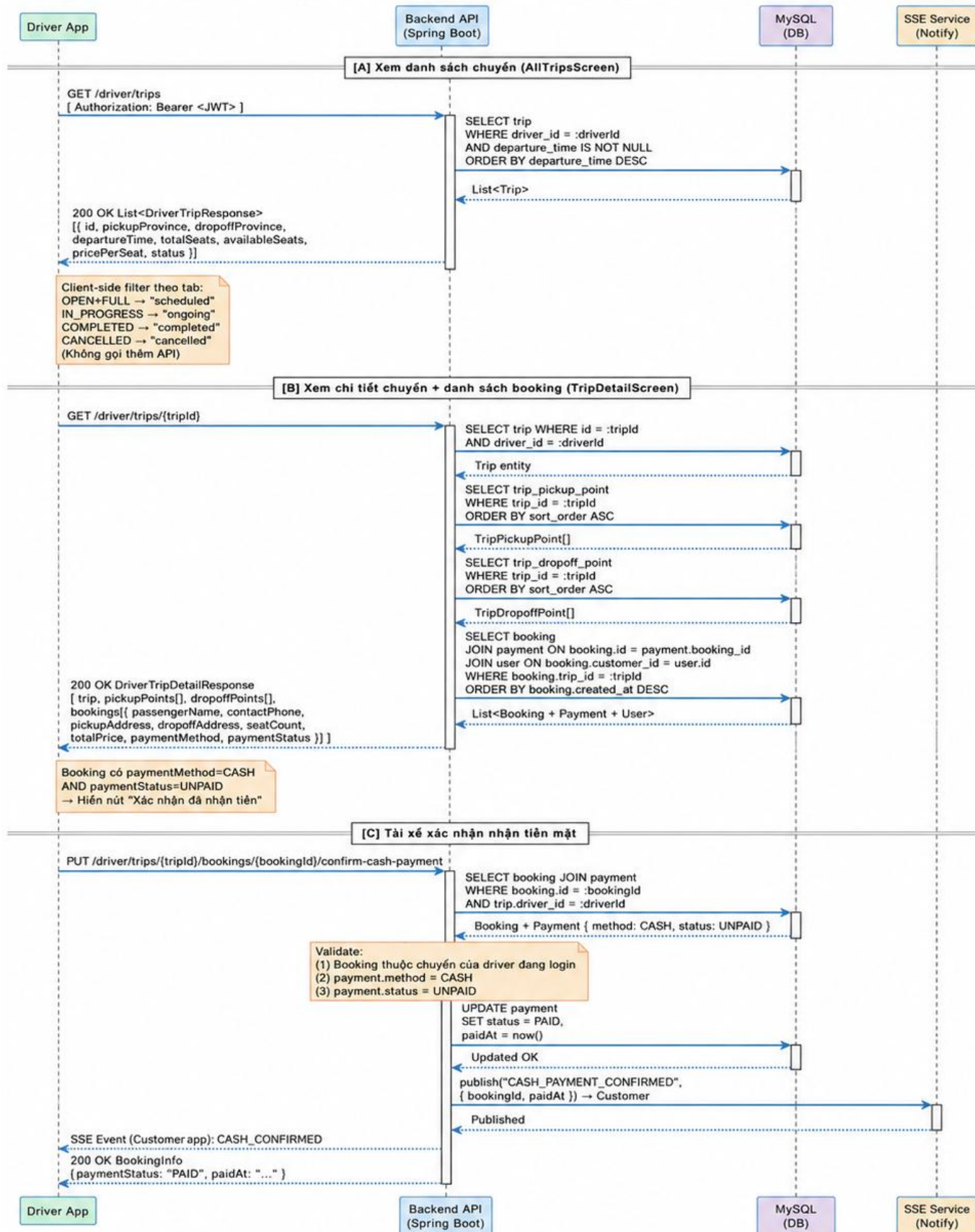
Hình 3.6 – Biểu đồ tuần tự: Bắt đầu và hoàn thành chuyến

3.5.4. Luồng hủy chuyến và hoàn tiền tự động



Hình 3.7 – Biểu đồ tuần tự: Hủy chuyến và hoàn tiền tự động

3.5.5. Luồng xem booking và xác nhận thanh toán tiền mặt



Hình 3.8 – Biểu đồ tuần tự: Xem booking và xác nhận thanh toán tiền mặt

3.6. Thiết kế giao diện và luồng màn hình

Giao diện module Tài xế được xây dựng theo theme màu **xanh lá (#00B14F)**, nhất quán qua tất cả các màn hình. Dưới đây là mô tả các màn hình chính:

Màn hình	Component File	Chức năng và thành phần UI chính
Driver Home	DriverHomeScreen.js	Header với tên tài xế; thống kê tháng (số chuyến, doanh thu, rating); danh sách 5 chuyến gần nhất; FAB "Tạo chuyến mới"; skeleton loading khi đang tải; realtime refresh khi nhận SSE
Tạo chuyến	CreateTripScreen.js	ProvincePicker (chọn tỉnh xuất phát/đến); WardPicker (chọn khu vực đón/trả dạng chip); SeatStepper (+- ghế 1-8); TextInput giá vé; CalendarPicker ngày; TimeSlot picker giờ (48 slots 30 phút); RevenuePreview card; nút submit với loading
Tất cả chuyến	AllTripsScreen.js	Tab filter (Tất cả/Lên lịch/Đang chạy/Hoàn thành/Đã hủy); TripCard với progress bar ghế đã đặt; nút Bắt đầu/Hoàn thành/Hủy trực tiếp trên card; Modal xác nhận hủy với input lý do
Chi tiết chuyến	TripDetailScreen.js	Header với trạng thái badge; thông tin tuyến và giờ; section điểm đón/trả theo thứ tự; thống kê ghế; danh sách BookingCard với đầy đủ thông tin hành khách; nút xác nhận tiền mặt; nút Bắt đầu/Hoàn thành/Hủy
Hồ sơ tài xế	DriverProfileScreen.js	Card thông tin cá nhân (họ tên, SĐT, email, ngày sinh); card tài liệu (CCCD mặt trước/sau, GPLX); card thông tin xe (biển số, hãng, dòng, năm, màu, số chỗ); upload ảnh qua ImagePicker; nút Lưu và nút Nộp hồ sơ

CHƯƠNG 4. KẾT QUẢ THỰC HIỆN

4.1. Mô hình triển khai ứng dụng

Hệ thống Ride Up được triển khai trên nền tảng cloud theo mô hình sau:

Thành phần	Nền tảng	Ghi chú kỹ thuật
Backend Spring Boot	[Railway / Render / VPS]	Java 17, Maven; expose REST API trên cổng 8080; kết nối MySQL, MongoDB, Redis đều qua biến môi trường
MySQL	Supabase (managed)	MySQL 15; connection pooling qua Supabase Pooler; dữ liệu được backup tự động
MongoDB	MongoDB Atlas (M0 free)	Lưu trữ ChatThreadDocument và ChatMessageDocument; index trên threadId và createdAt
Redis	[Upstash / Railway Redis]	TTL cache cho API responses; lưu blacklisted refresh tokens; prefix keys phân loại theo module
File Storage	Supabase Storage	Bucket riêng cho từng loại: avatars, cccd-images, vehicle-images; public URL; max 50MB/file
Mobile App	Expo Go / APK build	React Native 0.74+; chạy trên Android API 26+ và iOS 12+; kết nối backend qua BASE_URL trong config.js

Bảng 4.1 – Mô hình triển khai ứng dụng

4.2. Các bước cài đặt và triển khai

4.2.1. Backend (Spring Boot)

1. Yêu cầu: Java 17+, Maven 3.8+
2. Clone source code: `git clone https://github.com/hungnx-yb/Ride_Up.git`
3. Vào thư mục backend: `cd Ride_Up/RideUp`
4. Cấu hình file `src/main/resources/application.yml` với các biến môi trường:

```
datasource:
  url: jdbc:mysql://skibidi-
rideup.j.aivencloud.com:22897/rideUp?sslMode=REQUIRED
  username: avnadmin
  password: AVNS_tDOZw5pGeL-7Zt1yVSW
data:
  redis:
```

```

host: rideupredis-rideup.j.aivencloud.com
port: 22898
username: default
password: AVNS_Ora9009Vimx6B0m86NU
timeout: 10s
ssl:
  enabled: true
mongodb:
  uri:
mongodb+srv://rideup_app:RideUpMongo12345@rideupcluster.xffguvv.mongodb.net/rideup_chat?retryWrites=true&w=majority&appName=RideUpCluster
  auto-index-creation: true
vnpay:
  tmn-code: ${VNPAY_TMN_CODE:HUSN60FW}
  hash-secret: ${VNPAY_HASH_SECRET:LJW9JMRPZ55WB9E7XOKK3GZ6TADT2DLQ}
  pay-url: ${VNPAY_PAY_URL:https://sandbox.vnpayment.vn/paymentv2/vpcpay.html}
  return-url:
${VNPAY_RETURN_URL:http://172.20.10.6:8080/rideUp/payments/vnpay/return}
  order-type: ${VNPAY_ORDER_TYPE:other}
  locale: ${VNPAY_LOCALE:vn}
  expire-minutes: ${VNPAY_EXPIRE_MINUTES:15}

```

5. Chạy ứng dụng: `mvn spring-boot:run`
6. Backend khởi động tại `http://localhost:8080`

4.2.2. Mobile App (React Native / Expo)

7. Yêu cầu: Node.js 18+, npm/yarn, Expo CLI (`npm install -g expo-cli`)
8. Vào thư mục frontend: `cd Ride_Up/RideUpUI`
9. Cài dependencies: `npm install`
10. Cấu hình `BASE_URL` trong `src/config/config.js`:

```

export const BASE_URL = 'http://<your-backend-ip>:8080';
export const COLORS = { primary: '#00B14F', secondary: '#008A3E' };

```
11. Chạy development server: `npx expo start`
12. Quét QR bằng ứng dụng Expo Go (Android/iOS) hoặc nhấn A để mở Android Emulator
13. Build APK: `eas build -p android --profile preview`

4.3. Kết quả thực hiện – Các chức năng Module Tài xế

4.3.1. Chức năng Đăng ký và cập nhật hồ sơ tài xế

Màn hình hồ sơ tài xế (DriverProfileScreen) hiển thị form đầy đủ được chia thành ba nhóm thông tin: thông tin cá nhân, tài liệu pháp lý (CCCD, GPLX) và thông tin phương tiện. Tài xế có thể tải ảnh lên trực tiếp từ thư viện ảnh điện thoại thông qua ImagePicker. Khi điền đủ thông tin, nút "Nộp hồ sơ" được kích hoạt.

Sau khi nộp hồ sơ thành công, toàn bộ form bị khóa không cho sửa và hiển thị trạng thái "Chờ duyệt". Khi Admin duyệt hoặc từ chối, tài xế nhận thông báo realtime qua SSE.

Quản lý thông tin tài xế
Trạng thái: Đã duyệt ⭐ 5 · 0 chuyến

Thông tin cá nhân

Họ và tên: Trần Lê Lướt

Số điện thoại: 09xxxxxxxx

Email: dri@gmail.com

Ngày sinh: 2003-04-01

Avatar URL: 4-bacb-3616c77694b0-Screenshot 2026-04-02 082228.png

Giấy tờ tài xế

CCCD: 008200325687

Ảnh CCCD mặt trước: d11-Screenshot 2025-04-19 153909.png [Tải ảnh]

Ảnh CCCD mặt sau: 77d-Screenshot 2025-04-19 153909.png [Tải ảnh]

GPLX: B2

Hạn GPLX: 2029-04-30

Ảnh GPLX: 41b-Screenshot 2025-04-19 153909.png [Tải ảnh]

Thông tin xe

Biển số: 30A-36363

Hãng xe: Toyota

Dòng xe: Vios

Năm xe:

Số ghế:

Thanh menu: Trang chủ, Chuyến xe, Tin nhắn, Thông báo, Tài khoản

Hình 4.2, 4.3 – Màn hình Hồ sơ tài xế

4.3.2. Chức năng Tạo chuyến đi

Màn hình CreateTripScreen cung cấp một form trực quan để tài xế tạo chuyến mới hoàn toàn từ đầu (không còn dựa vào tuyến template). Tài xế chọn tỉnh xuất phát và tỉnh đến qua ProvincePicker, sau đó chọn các khu vực đón/trả

trong tính tương ứng qua WardPicker (hiển thị dạng chip, có thể thêm nhiều vùng).

Giá vé được nhập dạng text với format tiền Việt Nam. Số ghế được điều chỉnh qua SeatStepper (nút +/-). Ngày và giờ được chọn qua Calendar Picker và Time Slot grid (48 slot, mỗi slot 30 phút). Phần dưới màn hình hiển thị "Doanh thu dự kiến" được tính toán realtime.

16:50 • 📶 94

< Tạo chuyến xe

1. Tuyến đường

Tỉnh/TP đón *

Chọn tỉnh đón >

Xã/phường đón *

Chọn tỉnh đón trước >

Tỉnh/TP trả *

Chọn tỉnh trả >

Xã/phường trả *

Chọn tỉnh trả trước >

Giá vé cố định (đ) *

Ví dụ: 120000

2. Lịch khởi hành

Ngày (dd/MM/yyyy) *

Chọn ngày 📅

Giờ (HH:MM) *

Chọn giờ 🕒

3. Số ghế

- 4 +
ghế

4. Ghi chú (tùy chọn)

Ví dụ: Có điều hòa, đón linh hoạt...

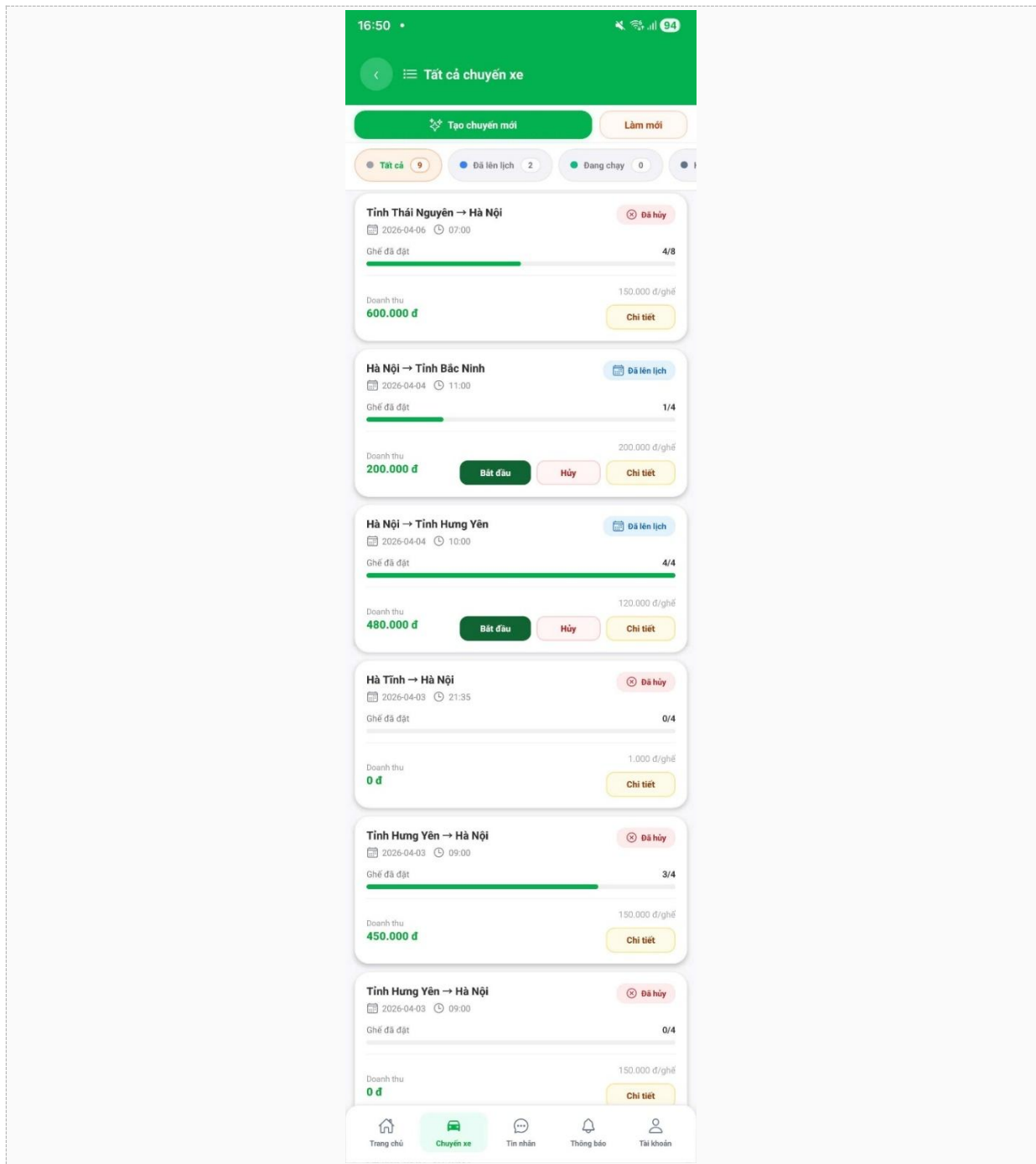
Lưu và tạo chuyến

Trang chủ **Chuyến xe** Tin nhắn Thông báo Tài khoản

Hình 4.4 – Màn hình Tạo chuyến đi

4.3.3. Chức năng Xem danh sách chuyến

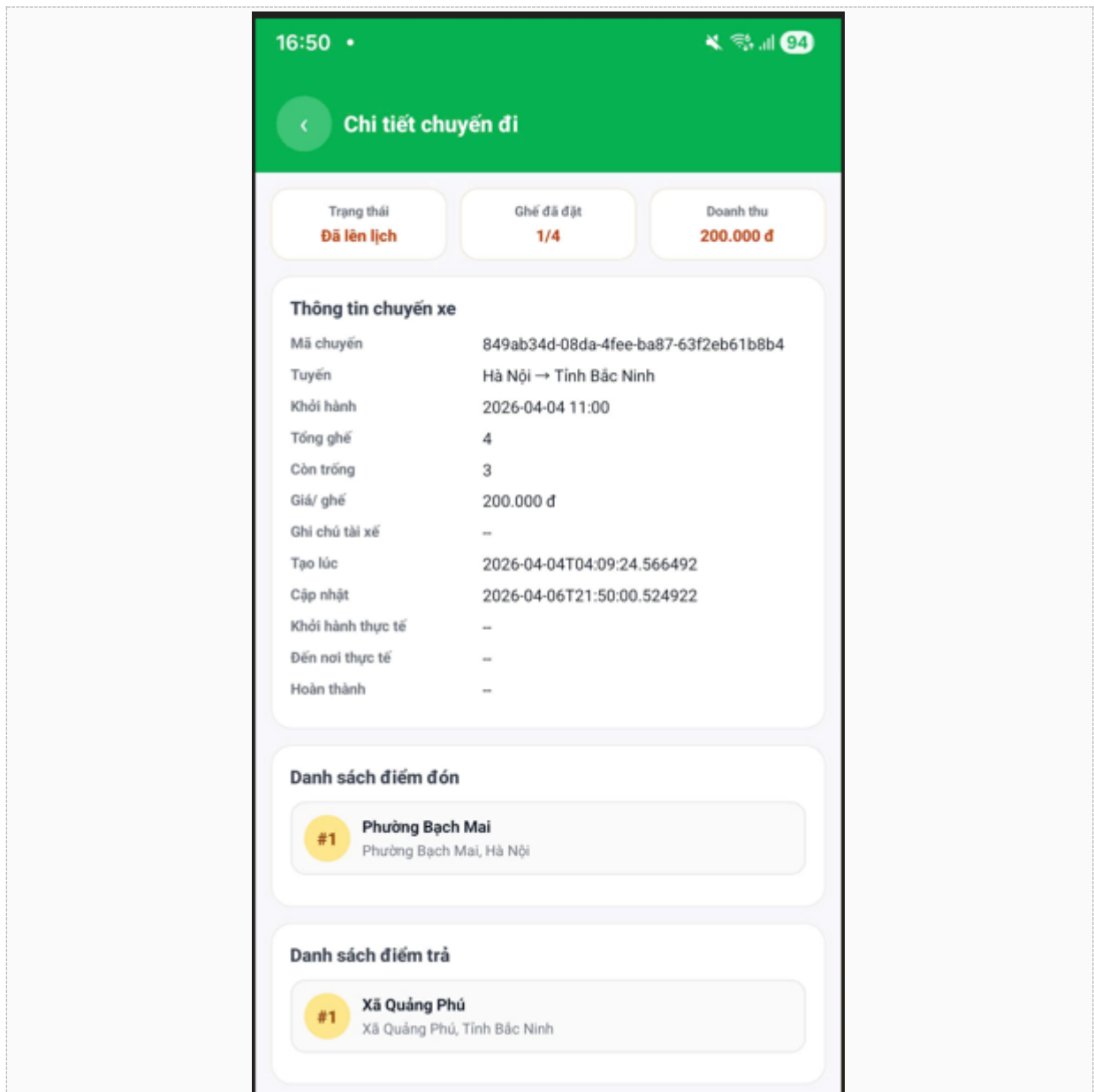
Màn hình AllTripsScreen hiển thị toàn bộ danh sách chuyến của tài xế với hệ thống lọc theo tab trạng thái. Mỗi TripCard hiển thị tuyến đường (tỉnh A → tỉnh B), ngày giờ khởi hành, badge trạng thái với màu riêng biệt, và thanh progress bar thể hiện tỷ lệ ghế đã đặt. Từ card, tài xế có thể thực hiện ngay các thao tác Bắt đầu / Hoàn thành / Hủy mà không cần vào chi tiết.



Hình 4.5 – Màn hình Danh sách chuyến đi (AllTrips)

4.3.4. Chức năng Xem chi tiết chuyến

Màn hình TripDetailScreen hiển thị đầy đủ thông tin chuyến đi: thông tin tuyến, ngày giờ lên lịch và thực tế, thống kê ghế (tổng/đã đặt/còn trống), doanh thu dự kiến, danh sách điểm đón và trả theo thứ tự. Các nút hành động (Bắt đầu/Hoàn thành/Hủy) hiển thị tùy theo trạng thái hiện tại của chuyến.



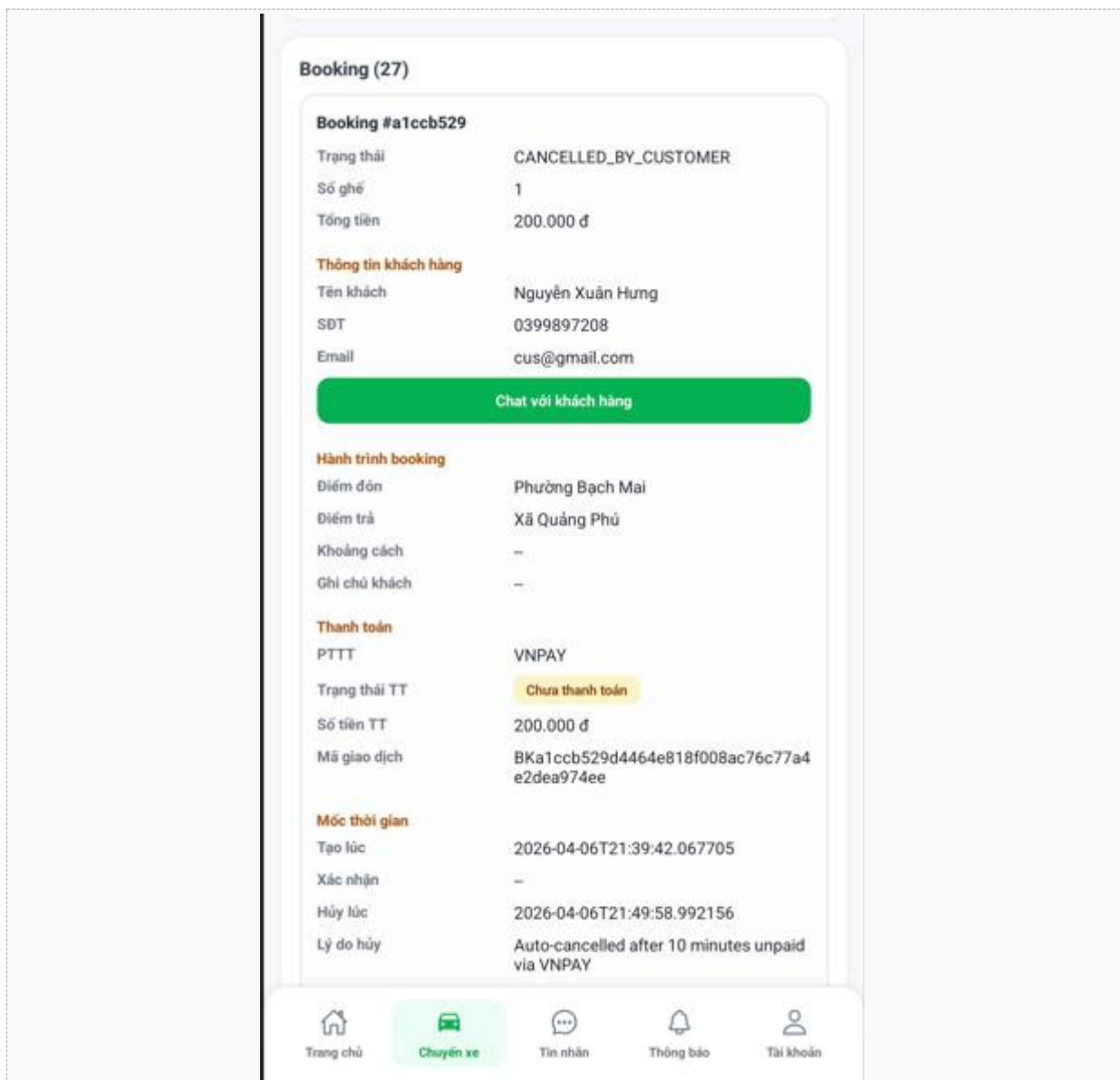
Hình 4.6 – Màn hình Chi tiết chuyến đi

4.3.5. Chức năng Xem danh sách booking

Trong màn hình chi tiết chuyến, phần booking hiển thị danh sách tất cả hành khách đã đặt chỗ. Mỗi BookingCard hiển thị: ảnh avatar hành khách, họ tên,

số điện thoại liên hệ, ghi chú hành khách, địa chỉ đón và trả cụ thể, số ghế, tổng tiền, phương thức và trạng thái thanh toán.

Với các booking thanh toán tiền mặt (CASH) chưa xác nhận (UNPAID), hệ thống hiển thị nút "Xác nhận đã nhận tiền" màu xanh. Khi tài xế bấm xác nhận, hệ thống cập nhật payment.status = PAID và gửi thông báo realtime đến hành khách ngay lập tức. Tính năng này bảo vệ tài xế khỏi việc hoàn thành chuyến mà chưa thu tiền.



Hình 4.7 – Màn hình Danh sách Booking

4.4. Kết quả thử nghiệm

Chức năng	Kịch bản kiểm thử	Kết quả	Ghi chú
Đăng ký hồ sơ	Upload ảnh CCCD, GPLX, xe lên Supabase Storage	Đạt	URL ảnh lưu thành công vào DB
Validate submit hồ sơ	Nộp hồ sơ thiếu CCCD/biển số	Đạt	API trả 400 DRIVER_PROFILE_INCOMPLETE
Hồ sơ bị khóa khi chờ duyệt	Cập nhật sau khi nộp hồ sơ	Đạt	API trả 400 DRIVER_PROFILE_LOCKED
Tạo chuyến đi	Tạo chuyến với đầy đủ thông tin hợp lệ	Đạt	Trip lưu DB, SSE tới tài xế tức thì
Validate tạo chuyến	Tạo chuyến khi hồ sơ chưa APPROVED	Đạt	Redirect sang DriverProfileScreen
Bắt đầu chuyến	Bắt đầu trước giờ lên lịch	Đạt	API trả 400 TRIP_START_BEFORE_SCHEDULE
Bắt đầu chuyến	Bắt đầu đúng giờ/sau giờ	Đạt	actualDepartureTime ghi lại chính xác
Hoàn thành chuyến	Còn booking tiền mặt chưa xác nhận	Đạt	API trả 400 UNCONFIRMED_CASH_PAYMENTS
Hoàn thành chuyến	Tất cả booking đã xác nhận/VNPay	Đạt	Booking→COMPLETED, ChatThreads đóng
Hủy chuyến + hoàn tiền	Hủy chuyến có booking VNPay	Đạt	VNPay refund tự động; khách nhận SSE
Hủy chuyến + hoàn tiền	VNPay refund thất bại (timeout)	Đạt	Log warning, chuyến vẫn hủy thành công
Xác nhận tiền mặt	Xác nhận booking CASH/UNPAID	Đạt	Payment→PAID, SSE đến khách tức thì
Xác nhận tiền mặt	Xác nhận booking VNPay (sai loại)	Đạt	API trả 400 PAYMENT_CONFIRM_NOT_ALLOWED

Thống kê doanh thu	Tính doanh thu tháng với chuyến đã hoàn thành	Đạt	Tính chính xác: $\text{price} \times \text{bookedSeats}$
--------------------	---	-----	--

Bảng 4.2 – Kết quả kiểm thử các chức năng Module Tài xế

4.5. Dữ liệu thử nghiệm trong hệ thống

Loại dữ liệu	Số lượng
Số tài khoản đã đăng ký	12
Số hồ sơ tài xế đã duyệt (APPROVED)	5
Số chuyến đi đã tạo	36
Số chuyến đã hoàn thành	33
Số booking đã tạo	54
Số tỉnh/thành phố trong hệ thống	34
Số xã/phường trong hệ thống	3321

Bảng 4.3 – Dữ liệu thử nghiệm trong hệ thống

CHƯƠNG 5. KẾT LUẬN

5.1. Kết quả đạt được

Trong phạm vi cá nhân phụ trách module Tài xế, bài tập lớn đã hoàn thành được các kết quả sau:

- Xây dựng đầy đủ luồng đăng ký, cập nhật hồ sơ tài xế với xác minh qua Admin và hệ thống khóa/mở hồ sơ
- Triển khai chức năng tạo chuyến đi với đầy đủ validation: kiểm tra hồ sơ được duyệt, kiểm tra giá vé tối thiểu, kiểm tra ngày không trong quá khứ, resolver ward IDs từ tên khu vực
- Xây dựng luồng quản lý trạng thái chuyến đầy đủ và chặt chẽ: OPEN → IN_PROGRESS → COMPLETED/CANCELLED với các guard condition ở mỗi bước
- Tích hợp hoàn tiền tự động VNPay khi tài xế hủy chuyến, với xử lý lỗi graceful (refund thất bại không ảnh hưởng đến việc hủy chuyến)
- Xây dựng chức năng xác nhận thanh toán tiền mặt với guard chống trùng lặp và thông báo realtime
- Triển khai hệ thống thông báo realtime SSE cho tất cả các sự kiện quan trọng trong vòng đời chuyến đi
- Xây dựng giao diện React Native đồng nhất, thân thiện với UX tốt: skeleton loading, animation entrance, success overlay, progress bar ghế
- API thiết kế RESTful rõ ràng, có phân quyền JWT, xử lý lỗi tập trung qua GlobalExceptionHandler

5.2. Hạn chế và điểm cần cải thiện

- Chức năng tracking GPS realtime (theo dõi vị trí tài xế trong khi chuyến đang diễn ra) chưa được triển khai trong phạm vi cá nhân
- Chưa có chức năng tái tạo chuyến nhanh (repeat trip) cho các tuyến cố định hàng ngày

- Thống kê doanh thu hiện chỉ hiển thị theo tháng hiện tại, chưa có tùy chọn lọc theo khoảng thời gian tùy chọn
- Chưa có chức năng chỉnh sửa chuyến đi sau khi tạo (chỉ có hủy), gây bất tiện khi muốn thay đổi giờ giấc

5.3. Hướng phát triển

- Tích hợp Google Maps hoặc OpenStreetMap để hiển thị lộ trình và tracking GPS trực quan trên bản đồ
- Thêm chức năng chỉnh sửa chuyến đi (thay đổi giờ, số ghế, giá) trước khi có booking
- Xây dựng tính năng Repeat Trip: tạo lịch chuyến định kỳ (hàng tuần, thứ cố định) từ một chuyến đã tạo
- Bổ sung thống kê nâng cao: biểu đồ doanh thu theo tuần/tháng/năm, tỷ lệ đánh giá sao, tuyến đường phổ biến nhất
- Tích hợp thêm cổng thanh toán MoMo, ZaloPay để tăng lựa chọn cho hành khách
- Thêm tính năng thông báo Push Notification (FCM) để tài xế nhận thông báo ngay cả khi app đang chạy nền

TÀI LIỆU THAM KHẢO

- 14.Spring Boot Reference Documentation. (2024).
<https://docs.spring.io/spring-boot/docs/3.2.x/reference/html/>
- 15.Spring Security Reference. (2024). <https://docs.spring.io/spring-security/reference/>
- 16.Spring Data JPA Reference Documentation. <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>
- 17.React Native Documentation. (2024). <https://reactnative.dev/docs/getting-started>
- 18.Expo Documentation. (2024). <https://docs.expo.dev/>
- 19.Supabase Documentation – Database & Storage.
<https://supabase.com/docs>
- 20.MongoDB Atlas Documentation. <https://www.mongodb.com/docs/atlas/>
- 21.Redis Documentation. <https://redis.io/docs/>
- 22.VNPay Developer Documentation.
<https://sandbox.vnpayment.vn/apis/docs/thanh-toan-pay/api.html>
- 23.JSON Web Tokens (JWT). <https://jwt.io/introduction/>
- 24.Project Lombok Documentation. <https://projectlombok.org/features/>
- 25.ModelMapper Documentation. <http://modelmapper.org/>
- 26.Postman API Testing. <https://www.postman.com/>
- 27.OODesign – Design Patterns. <https://www.oodesign.com/>
- 28.Baeldung Spring Boot Tutorials. <https://www.baeldung.com/spring-boot>

PHỤ LỤC I. CODE ĐÁP ỨNG CHỨC NĂNG

Chương này trình bày chi tiết các lớp, hàm, bảng CSDL và API gọi ngoài phục vụ module Role Tài xế. Toàn bộ code nằm trong project Spring Boot (backend) và React Native (mobile app).

1.1. Các lớp Entity và Bảng trong CSDL

Tất cả entity đều sử dụng UUID làm khóa chính, có các cột audit created_at/updated_at và được ánh xạ sang bảng MySQL qua Spring Data JPA.

1.1.1. Entity User – Bảng user

File: RideUp/src/main/java/com/example/demo/entity/User.java

Mô tả: Lưu thông tin tài khoản của tất cả người dùng (Driver, Customer, Admin). Tài xế sử dụng bảng này để xác thực JWT và lấy thông tin cá nhân.

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	VARCHAR (UUID)	PK	Khóa chính, tự sinh UUID
full_name	VARCHAR	NOT NULL	Họ tên đầy đủ của người dùng
phone_number	VARCHAR	UNIQUE	Số điện thoại (dùng làm username đăng nhập)
email	VARCHAR	UNIQUE, NOT NULL	Địa chỉ email
password	VARCHAR	NOT NULL	Mật khẩu đã mã hóa BCrypt
roles	ENUM collection		Tập hợp role: DRIVER, CUSTOMER, ADMIN
avatar_url	VARCHAR		URL ảnh đại diện trên Supabase Storage
date_of_birth	DATE		Ngày sinh
gender	ENUM		MALE, FEMALE, OTHER
created_at	TIMESTAMP	NOT NULL	Thời điểm tạo tài khoản

1.1.2. Entity DriverProfile – Bảng driver_profile

File: RideUp/src/main/java/com/example/demo/entity/DriverProfile.java

Mô tả: Lưu hồ sơ xác minh tài xế, liên kết 1-1 với bảng user. Quản lý trạng thái duyệt hồ sơ và rating của tài xế.

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	VARCHAR (UUID)	PK	Khóa chính
user_id	VARCHAR	FK→user, UNIQUE	Liên kết 1-1 với bảng user
cccd	VARCHAR		Số căn cước công dân
cccd_image_front	VARCHAR		URL ảnh CCCD mặt trước (Supabase Storage)
cccd_image_back	VARCHAR		URL ảnh CCCD mặt sau
gplx	VARCHAR		Số giấy phép lái xe
gplx_expiry_date	DATE		Ngày hết hạn GPLX
gplx_image	VARCHAR		URL ảnh GPLX (Supabase Storage)
driver_rating	DOUBLE	DEFAULT 0.0	Điểm đánh giá trung bình (0-5)
total_driver_rides	INT	DEFAULT 0	Tổng số chuyến đã hoàn thành
status	ENUM	NOT NULL	PENDING APPROVED REJECTED
submitted	BOOLEAN	DEFAULT false	true khi đã nộp hồ sơ (hồ sơ bị khóa chỉnh sửa)
approved_at	TIMESTAMP		Thời điểm Admin duyệt hồ sơ
approved_by	VARCHAR		ID Admin đã duyệt
rejected_at	TIMESTAMP		Thời điểm bị từ chối
rejection_reason	TEXT		Lý do từ chối của Admin

1.1.3. Entity Vehicle – Bảng vehicle

File: RideUp/src/main/java/com/example/demo/entity/Vehicle.java

Mô tả: Lưu thông tin phương tiện của tài xế, liên kết 1-1 với driver_profile.

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	VARCHAR (UUID)	PK	Khóa chính
driver_id	VARCHAR	FK→driver_profile, UNIQUE	Liên kết 1-1 với driver_profile
plate_number	VARCHAR		Biển số xe (bắt buộc khi nộp hồ sơ)
vehicle_brand	VARCHAR		Hãng xe (Toyota, Honda...)
vehicle_model	VARCHAR		Dòng xe (Vios, City...)
vehicle_year	INT		Năm sản xuất
vehicle_color	VARCHAR		Màu xe
seat_capacity	INT		Số ghế tối đa của xe
vehicle_type	ENUM		Loại xe: CAR, TRUCK, VAN...
vehicle_image	VARCHAR		URL ảnh xe
registration_image	VARCHAR		URL ảnh đăng ký xe
registration_expiry_date	DATE		Ngày hết hạn đăng ký xe
insurance_image	VARCHAR		URL ảnh bảo hiểm xe
insurance_expiry_date	DATE		Ngày hết hạn bảo hiểm
is_verified	BOOLEAN	DEFAULT false	Admin đã xác minh xe chưa
is_active	BOOLEAN	DEFAULT true	Xe có đang hoạt động không

1.1.4. Entity Trip – Bảng trip

File: RideUp/src/main/java/com/example/demo/entity/Trip.java

Mô tả: Lưu thông tin chuyến đi của tài xế. Mỗi trip có nhiều điểm đón (TripPickupPoint), nhiều điểm trả (TripDropoffPoint) và nhiều booking. Bảng có 3 index phức hợp tối ưu truy vấn theo driver_id và departure_time.

Trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	VARCHAR (UUID)	PK	Khóa chính
driver_id	VARCHAR	FK→driver_profile	Tài xế tạo chuyến (N-1)
departure_time	DATETIME		Thời gian khởi hành đã lên lịch (NULL = template)
total_seats	INT		Tổng số ghế cung cấp
available_seats	INT		Số ghế còn trống (giảm khi booking xác nhận)
price_per_seat	DECIMAL (10,2)		Giá mỗi ghế (đồng VND)
status	ENUM		OPEN FULL IN_PROGRESS COMPLETED CANCELLED
driver_note	TEXT		Ghi chú của tài xế cho chuyến
actual_departure_time	DATETIME		Thời điểm thực tế xuất phát (khi driver bấm Bắt đầu)
actual_arrival_time	DATETIME		Thời điểm thực tế đến nơi
completed_at	DATETIME		Thời điểm chuyển chuyển sang COMPLETED

Index CSDL:

```
@Index(name = "idx_trip_status_departure", columnList =
"status,departure_time")
@Index(name = "idx_trip_departure", columnList = "departure_time")
@Index(name = "idx_trip_driver_departure", columnList =
"driver_id,departure_time")
```

1.1.5. Entity TripPickupPoint & TripDropoffPoint

File: entity/TripPickupPoint.java, entity/TripDropoffPoint.java

Mô tả: Lưu các điểm đón/trả của một chuyến đi, liên kết với Ward (xã/phường). sortOrder xác định thứ tự đón/trả trên lộ trình.

Trường	Kiểu	Mô tả
id	VARCHAR (UUID)	Khóa chính
trip_id	VARCHAR	FK→trip
district_id (ward_id)	VARCHAR	FK→ward – xã/phường đón/trả
address	VARCHAR	Địa chỉ cụ thể tại điểm đón/trả
pickup_time / dropoff_time	TIME	Giờ đón/trả dự kiến
sort_order	INT	Thứ tự trong lộ trình (0, 1, 2...)
note	VARCHAR	Ghi chú thêm của tài xế

1.1.6. Entity Booking – Bảng booking

File: RideUp/src/main/java/com/example/demo/entity/Booking.java

Mô tả: Lưu thông tin đặt chỗ của hành khách trên chuyến đi. Có 3 index phức hợp tối ưu truy vấn theo customer_id, trip_id và status.

Trường	Kiểu dữ liệu	Mô tả
id	VARCHAR (UUID)	Khóa chính
trip_id	VARCHAR FK	Chuyến xe (N-1)
customer_id	VARCHAR FK	Khách đặt chỗ (N-1)
pickup_point_id	VARCHAR FK	Điểm đón đã chọn
dropoff_point_id	VARCHAR FK	Điểm trả đã chọn
pickup_address / dropoff_address	VARCHAR	Địa chỉ cụ thể đón/trả
pickup_lat, pickup_lng	DOUBLE	Tọa độ điểm đón (khách pin trên map)
seat_count	INT NOT NULL	Số ghế đặt
total_price	DECIMAL(10,2)	Tổng tiền = seatCount × pricePerSeat

Trường	Kiểu dữ liệu	Mô tả
status	ENUM	PENDING CONFIRMED COMPLETED CANCELLED_BY_CUSTOMER CANCELLED_BY_DRIVER NO_SHOW
passenger_name	VARCHAR	Tên người đi (có thể khác tên tài khoản)
contact_phone	VARCHAR	Số điện thoại liên hệ
customer_note	TEXT	Ghi chú của khách
confirmed_at / cancelled_at / completed_at	DATETIME	Mốc thời gian chuyển trạng thái
cancellation_reason	TEXT	Lý do hủy

1.1.7. Entity Payment – Bảng payment

File: RideUp/src/main/java/com/example/demo/entity/Payment.java

Mô tả: Lưu thông tin thanh toán cho từng booking (1-1 với Booking). Hỗ trợ CASH và VNPay. Khi hoàn tiền, trường refunded_at và provider_transaction_id được cập nhật.

Trường	Kiểu	Mô tả
id	VARCHAR (UUID)	Khóa chính
booking_id	VARCHAR FK	Liên kết 1-1 với booking (UNIQUE)
amount	DECIMAL(10,2)	Số tiền thanh toán
method	ENUM	CASH BANK_TRANSFER VNPAY
status	ENUM	UNPAID PAID REFUNDED FAILED
transaction_id	VARCHAR	Mã giao dịch nội bộ
provider_transaction_id	VARCHAR	Mã giao dịch từ VNPay
paid_at	DATETIME	Thời điểm thanh toán thành công
refunded_at	DATETIME	Thời điểm hoàn tiền

1.1.8. Entity Ward & Province – Bảng ward, province

Mô tả: Lưu dữ liệu hành chính địa lý Việt Nam. Hệ thống có 34 tỉnh/thành phố và 3.321 xã/phường phục vụ chọn điểm đón/trả. Ward liên kết N-1 với Province.

1.2. Các Controller – Lớp xử lý HTTP Request

1.2.1. DriverTripController

File: RideUp/src/main/java/com/example/demo/controller/DriverTripController.java

Basepath: /driver

Annotation: @RestController, @PreAuthorize("isAuthenticated()")

HTTP Method	Endpoint	Hàm xử lý	Chức năng
GET	/driver/trips	getDriverTrips()	Lấy danh sách tất cả chuyến của tài xế hiện tại, sắp xếp theo departureTime giảm dần
POST	/driver/trips	createTrip(request)	Tạo chuyến đi mới, nhận DriverTripRequest, trả DriverTripResponse
GET	/driver/trips/{tripId}	getTripDetail(tripId)	Lấy chi tiết chuyến bao gồm danh sách điểm đón/trả và danh sách booking
PUT	/driver/trips/{tripId}/cancel	cancelTrip(tripId, request, ip)	Hủy chuyến, tự động hủy booking và refund VNPAY, đóng chat thread
PUT	/driver/trips/{tripId}/start	startTrip(tripId)	Bắt đầu chuyến (OPEN/FULL → IN_PROGRESS), kiểm tra giờ khởi hành
PUT	/driver/trips/{tripId}/complete	completeTrip(tripId)	Hoàn thành chuyến, cập nhật booking → COMPLETED, đóng chat thread
PUT	/driver/trips/{tripId}/bookings/{bookingId}/confirm-cash-payment	confirmCashPayment(tripId, bookingId)	Xác nhận đã thu tiền mặt, cập nhật payment.status = PAID, gửi SSE thông báo khách

HTTP Method	Endpoint	Hàm xử lý	Chức năng
GET	/driver/stats	getDriverStats()	Thống kê doanh thu tháng hiện tại, số chuyển, rating

1.2.2. DriverProfileController

File: RideUp/src/main/java/com/example/demo/controller/DriverProfileController.java

Basepath: /driver/profile

Annotation: @RestController, @PreAuthorize("isAuthenticated()")

HTTP Method	Endpoint	Hàm xử lý	Chức năng
GET	/driver/profile	getMyProfile()	Lấy hồ sơ tài xế hiện tại (hoặc tạo mới nếu chưa có)
PUT	/driver/profile	updateMyProfile(request)	Cập nhật thông tin cá nhân, tài liệu pháp lý và thông tin xe. Hồ sơ đang PENDING sẽ bị lock.
POST	/driver/profile/submit	submitMyProfile(request)	Nộp hồ sơ để Admin duyệt. Validate bắt buộc: CCCD, GPLX, biển số, hãng xe, dòng xe. Chuyển status → PENDING, submitted = true.

1.2.3. FileController

File: RideUp/src/main/java/com/example/demo/controller/FileController.java

Base path: /file | Dùng để upload ảnh CCCD, GPLX, ảnh xe lên Supabase Storage.

HTTP Method	Endpoint	Hàm xử lý	Chức năng
POST	/file/upload	uploadFiles(file)	Nhận file multipart, upload lên Supabase Storage bucket, trả về đường dẫn object. Content-Type tự phát hiện từ file gốc.

1.2.4. LocationController

File:

RideUp/src/main/java/com/example/demo/controller/LocationController.java

Cung cấp dữ liệu địa lý phục vụ form tạo chuyến đi.

HTTP Method	Endpoint	Chức năng
GET	/locations/provinces	Trả danh sách toàn bộ tỉnh/thành phố trong hệ thống (34 tỉnh)
GET	/locations/wards?provinceId={id}	Trả danh sách xã/phường thuộc tỉnh được chọn

1.3. Các Service – Lớp xử lý nghiệp vụ

1.3.1. DriverTripService

File: RideUp/src/main/java/com/example/demo/service/DriverTripService.java

Đây là service trung tâm xử lý toàn bộ nghiệp vụ quản lý chuyến đi của tài xế. Tất cả phương thức đều được bao bọc trong @Transactional.

Các hàm chính:

Tên hàm	Annotation	Mô tả chi tiết
getMyTrips()	@Transactional(readonly=true)	Lấy danh sách chuyến của tài xế đang đăng nhập. Gọi getOrCreateDriverProfile() để lấy DriverProfile, sau đó query TripRepository với departureTime IS NOT NULL (loại bỏ template).
createTrip(request)	@Transactional	Tạo chuyến đi mới: (1) validate request, (2) kiểm tra hồ sơ APPROVED, (3) gọi resolveOrCreateTemplate() để tạo hoặc tái sử dụng route template, (4) tạo Trip entity với TripPickupPoints và TripDropoffPoints, (5) lưu DB, (6) publish SSE DRIVER_TRIP_CREATED đến tài xế.

Tên hàm	Annotation	Mô tả chi tiết
cancelTrip(tripId, request, ip)	@Transactional	Hủy chuyến: validate status NOT IN {COMPLETED, CANCELLED, IN_PROGRESS} → cập nhật trip. status = CANCELLED → gọi markBookingsCancelledByDriver() để hủy booking và refund VNPay → đóng chat thread → publish SSE đến tài xế và từng hành khách bị ảnh hưởng.
startTrip(tripId)	@Transactional	Bắt đầu chuyến: validate status IN {OPEN, FULL} → validate departureTime <= now() (nếu sớm hơn giờ lịch → 409 TRIP_START_BEFORE_SCHEDULE) → cập nhật status = IN_PROGRESS, actualDepartureTime = now().
completeTrip(tripId)	@Transactional	Hoàn thành chuyến: validate status == IN_PROGRESS → kiểm tra không còn booking CASH+UNPAID (nếu còn → 409 UNCONFIRMED_CASH_PAYMENTS) → cập nhật status = COMPLETED → gọi markBookingsCompleted() → đóng chat thread → publish SSE.
confirmCashPayment(tripId, bookingId)	@Transactional	Xác nhận thanh toán tiền mặt: validate booking thuộc trip của driver đang login, payment.method == CASH, payment.status == UNPAID → cập nhật payment.status = PAID, paidAt = now() → publish SSE CASH_PAYMENT_CONFIRMED đến hành khách.
getDriverStats()	@Transactional(readOnly=true)	Thống kê tháng hiện tại: đếm totalRides, completedRides, cancelledRides và tính revenue (price × bookedSeats) cho tất cả chuyến trong tháng. Trả thêm driverRating và totalReviews từ DriverProfile.

Tên hàm	Annotation	Mô tả chi tiết
getOrCreateDriverProfile()	private	Lấy DriverProfile của user hiện tại. Nếu chưa có → tạo mới với status=PENDING, submitted=false, driverRating=0.0. Xử lý fallback cho trường hợp có nhiều profile (legacy data).
resolveOrCreateTemplate(driver, request)	private	Tạo hoặc tái sử dụng route template: nếu request có routeId → load template từ DB; nếu không → resolve Province, resolve Wards từ tên ward, kiểm tra có template trùng route không, nếu có thì tái sử dụng, nếu không tạo mới.
markBookingsCancelledByDriver(trip, reason, ip)	private	Duyệt qua tất cả booking PENDING/CONFIRMED → cập nhật status = CANCELLED_BY_DRIVER, cancelledAt = now() → gọi customerBookingService. tryAutoRefundVnPay() để hoàn tiền. Lỗi refund được log nhưng không dừng luồng hủy chuyến.
markBookingsCompleted(trip, completedAt)	private	Duyệt qua tất cả booking CONFIRMED/PENDING → cập nhật status = COMPLETED, completedAt = now().
validateCreateRequest(request)	private	Validate form tạo chuyến: bắt buộc có departureDate, departureTime; nếu không có routeId thì phải có pickupProvince/dropoffProvince, pickupClusters/dropoffClusters, fixedFare >= 1000.
parseDepartureDateTime(date, time)	private	Phân tích ngày giờ linh hoạt: hỗ trợ cả định dạng ISO (yyyy-MM-dd) và Việt Nam (dd/MM/yyyy).

1.3.2. DriverProfileService

File:

RideUp/src/main/java/com/example/demo/service/DriverProfileService.java

Xử lý nghiệp vụ quản lý hồ sơ tài xế: tạo, cập nhật, nộp hồ sơ.

Tên hàm	Annotation	Mô tả chi tiết
getMyProfile()	@Transactional(readOnly=true)	Lấy hồ sơ tài xế hiện tại. Gọi getOrCreateDriverProfile() để tạo mới nếu chưa có, sau đó map sang DriverProfileResponse gồm thông tin User + DriverProfile + Vehicle.
updateMyProfile(request)	@Transactional	Cập nhật hồ sơ tài xế: (1) kiểm tra isProfileLocked() – nếu đang PENDING+submitted → trả 400 DRIVER_PROFILE_LOCKED; (2) cập nhật User (fullName, phone, dob, gender, avatarUrl); (3) cập nhật DriverProfile (cccd, gplx, gplxExpiryDate, images) (4) tạo hoặc cập nhật Vehicle entity; (5) nếu hồ sơ đang APPROVED và bị chỉnh sửa → tự động chuyển về PENDING để Admin duyệt lại.
submitMyProfile()	@Transactional	Nộp hồ sơ: gọi validateProfileForSubmit() kiểm tra bắt buộc có CCCD, GPLX, plateNumber, vehicleBrand, vehicleModel → cập nhật submitted=true, status=PENDING.
validateProfileForSubmit(profile)	private	Validate: hasCoreDocs = (cccd != null AND gplx != null); hasCoreVehicle = (plateNumber != null AND vehicleBrand != null AND vehicleModel != null). Ném DRIVER_PROFILE_INCOMPLETE nếu thiếu.
isProfileLocked(profile)	private	Hồ sơ bị khóa khi: submitted == true AND status == PENDING. Tài xế không thể chỉnh sửa khi đang chờ Admin duyệt.
toResponse(user, profile, vehicle)	private	Map User + DriverProfile + Vehicle sang DriverProfileResponse DTO. Bao gồm tất cả thông tin cá nhân, tài liệu pháp lý, thông tin xe và trạng thái hồ sơ.

1.3.3. FileService

File: RideUp/src/main/java/com/example/demo/service/FileService.java

Xử lý upload/delete file lên Supabase Storage qua REST API.

Tên hàm	Mô tả chi tiết
upload(file, prefix)	Upload file lên Supabase Storage: (1) tạo tên file an toàn = prefix/UUID-originalName (2) gửi HTTP POST đến Supabase REST API với header Authorization Bearer + apiKey (3) tự detect ContentType từ file, fallback về APPLICATION_OCTET_STREAM nếu invalid (4) dùng x-upsert: true để ghi đè nếu file đã tồn tại (5) trả về đường dẫn object (dùng lưu vào DB).
getFileUrl(objectPath)	Trả về Public URL của file trên Supabase: supabaseConfig.getPublicUrl(objectPath).
delete(objectPath)	Xóa file khỏi Supabase Storage qua HTTP DELETE, dùng trong trường hợp cần dọn dẹp file cũ.
sanitizeFileName(originalName)	Làm sạch tên file gốc, xóa ký tự đặc biệt (\\:*?"<>) bằng regex, trả fallback 'file.bin' nếu tên rỗng.

1.3.4. NotificationRealtimePublisher

File:

RideUp/src/main/java/com/example/demo/service/NotificationRealtimePublisher.java

Component phát sự kiện thông báo realtime đến client qua WebSocket (STOMP protocol).

Tên hàm	Mô tả chi tiết
notifyUser(targetUserId, type, title, message, referenceId)	Tạo RealtimeNotificationResponse với UUID mới, publish đến topic /topic/notifications.user.{targetUserId}. Sử dụng SimpMessagingTemplate của Spring WebSocket. Các loại type trong module Tài xế: DRIVER_TRIP_CREATED, DRIVER_TRIP_CANCELLED, DRIVER_TRIP_COMPLETED, TRIP_CANCELLED_BY_DRIVER, TRIP_COMPLETED, CASH_PAYMENT_CONFIRMED, DRIVER_PROFILE_APPROVED.

1.4. Các Repository – Lớp truy cập CSDL

Tất cả repository kế thừa JpaRepository<Entity, String> và sử dụng Spring Data JPA để tự sinh SQL. Một số query phức tạp dùng @Query JPQL.

1.4.1. TripRepository

File: RideUp/src/main/java/com/example/demo/repository/TripRepository.java

Tên hàm	Mô tả
findByDriverIdAndDepartureTimeIsNotNullOrderByDepartureTimeDesc(driverId)	Lấy tất cả chuyến (có departure_time) của tài xế, sắp xếp mới nhất lên đầu. Dùng trong getMyTrips().
findByIdAndDriverIdAndDepartureTimeIsNotNull(id, driverId)	Lấy chi tiết một chuyến, đảm bảo chuyến thuộc tài xế đang login. Dùng trong startTrip(), completeTrip().
findByIdAndDriverIdAndDepartureTimeIsNotNullWithBookings(tripId, driverId)	@Query JPQL với LEFT JOIN FETCH bookings và payment – tải eager để tránh N+1 query. Dùng trong cancelTrip() và confirmCashPayment().
findByDriverIdAndDepartureTimeIsNullOrderByUpdatedAtDesc(driverId)	Lấy danh sách route template (departure_time IS NULL) của tài xế để tái sử dụng khi tạo chuyến.
findByIdAndDriverIdAndDepartureTimeIsNull(id, driverId)	Lấy một route template cụ thể theo ID.
countByDriverId(driverId)	Đếm tổng số trip của một driver profile. Dùng để chọn profile chính khi có nhiều profile.
findByIdForUpdate(tripId)	@Lock(PESSIMISTIC_WRITE) – khóa pessimistic để tránh race condition khi nhiều request đồng thời cập nhật cùng một trip.
sumRevenueForPeriod(start, end, status)	@Query tính tổng doanh thu = SUM(pricePerSeat × bookedSeats) trong khoảng thời gian và trạng thái cho trước. Dùng trong admin dashboard.
countByCreatedAtBetween(start, end)	Đếm số chuyến được tạo trong khoảng thời gian.

1.4.2. DriverProfileRepository

File:

RideUp/src/main/java/com/example/demo/repository/DriverProfileRepository.java

Tên hàm	Mô tả
findByUserId(userId)	Tìm DriverProfile theo userId. Trả Optional để xử lý trường hợp chưa có hồ sơ.
findAllByUserIdOrderByCreatedAtDesc(userId)	Lấy tất cả profile của user (fallback xử lý data cũ), sắp xếp theo thời gian tạo mới nhất.

1.4.3. WardRepository

File: RideUp/src/main/java/com/example/demo/repository/WardRepository.java

Tên hàm	Mô tả
findByProvinceIdAndWardName(provinceId, wardName)	Tìm Ward theo tỉnh và tên xã/phường. Dùng trong resolveOrCreateTemplate() khi tạo chuyến.

1.5. Các DTO (Data Transfer Object)

DTO được dùng để tách biệt dữ liệu giữa tầng Controller và tầng Service, tránh expose trực tiếp Entity ra API.

Tên DTO	Loại	Mô tả
DriverTripRequest	Request	Body tạo chuyến: routeId (optional), pickupProvince, dropoffProvince, pickupClusters[], dropoffClusters[], fixedFare, totalSeats, availableSeats, departureDate (dd/MM/yyyy hoặc yyyy-MM-dd), departureTime (HH:mm), notes
TripCancellationRequest	Request	Body hủy chuyến: cancellationReason (optional)
DriverProfileUpdateRequest	Request	Body cập nhật hồ sơ: fullName, phoneNumber, dateOfBirth, gender, avatarUrl, cccd, cccdImageFront, cccdImageBack, gplx, gplxExpiryDate, gplxImage, plateNumber, vehicleBrand, vehicleModel, vehicleYear, vehicleColor, seatCapacity, vehicleType, vehicleImage, registrationImage, insuranceImage...
DriverTripResponse	Response	Thông tin tóm tắt một chuyến: id, routeId, pickupProvince, dropoffProvince, departureDate, departureTime, totalSeats, availableSeats, fixedFare, status (scheduled/ongoing/completed/cancelled)

Tên DTO	Loại	Mô tả
DriverTripDetailResponse	Response	Chi tiết chuyến: tất cả trường DriverTripResponse + createdAt, updatedAt, actualDepartureTime, actualArrivalTime, completedAt, bookedSeats, estimatedRevenue, driverNote + List<PointInfo> (pickupPoints, dropoffPoints) + List<BookingInfo>
DriverTripDetailResponse.PointInfo	Nested	Thông tin điểm đón/trả: id, wardName, provinceName, address, sortOrder, time, note
DriverTripDetailResponse.BookingInfo	Nested	Thông tin booking: id, status, seatCount, totalPrice, pickupAddress, dropoffAddress, passengerName, contactPhone, customerNote, customerId, customerName, customerPhone, customerAvatarUrl, paymentMethod, paymentStatus, paymentAmount, transactionId, paidAt, createdAt, confirmedAt, cancelledAt, cancellationReason, completedAt
DriverProfileResponse	Response	Hồ sơ tài xế đầy đủ: thông tin User + DriverProfile + Vehicle, thêm trường profileLocked (boolean) và submitted (boolean)
RealtimeNotificationResponse	Response/ WebSocket	Payload thông báo realtime: id, type, title, message, targetUserId, referenceId, createdAt

1.6. Các API Gọi Ngoài (External API)

1.6.1. Supabase Storage REST API

Mục đích: Lưu trữ ảnh CCCD, GPLX, ảnh xe của tài xế. Được gọi từ FileService.upload().

Cấu hình: SupabaseStorageConfig – đọc từ biến môi trường supabase.url, supabase.key, supabase.bucket.

Phương thức	Endpoint	Mô tả
POST	https://{project}.supabase.co/storage/v1/object/{bucket}/{fileName}	Upload file. Header: Authorization: Bearer {key},

Phương thức	Endpoint	Mô tả
		apikey: {key}, x-upsert: true. Body: binary file bytes.
DELETE	https://{project}.supabase.co/storage/v1/object/{bucket}/{objectPath}	Xóa file. Header: Authorization + apikey.
GET (public)	https://{project}.supabase.co/storage/v1/object/public/{bucket}/{objectPath}	Truy cập file công khai. URL này được lưu vào DB và trả về client.

1.6.2. VNPay Refund API

Mục đích: Hoàn tiền tự động cho hành khách khi tài xế hoặc khách hủy chuyến có thanh toán VNPay.

Được gọi từ CustomerBookingService.tryAutoRefundVnPay() (được invoke bởi DriverTripService.markBookingsCancelledByDriver()).

Thông số	Giá trị
URL Sandbox	https://sandbox.vnpayment.vn/merchant_webapi/api/transaction
Phương thức	POST
Tham số bắt buộc	vnp_TmnCode, vnp_TransactionDate, vnp_TransactionNo, vnp_Amount, vnp_OrderInfo, vnp_TransactionType=02 (hoàn tiền), vnp_CreateDate, vnp_IpAddr, vnp_SecureHash
Xử lý lỗi	Refund thất bại sẽ chỉ log warning, không dừng luồng hủy chuyến. payment.status giữ nguyên PAID nếu refund thất bại.
Kết quả thành công	vnp_ResponseCode = 00 → cập nhật payment.status = REFUNDED, refundedAt = now(), providerTransactionId = vnp_TransactionNo

1.6.3. WebSocket STOMP – Thông báo Realtime

Mục đích: Đẩy thông báo tức thì đến client mà không cần polling. Dùng SimpMessagingTemplate của Spring WebSocket.

Topic	Loại event	Trigger
/topic/notifications.user.{userId}	DRIVER_TRIP_CREATED	Sau khi tạo chuyến thành công

Topic	Loại event	Trigger
/topic/notifications.user.{userId}	DRIVER_TRIP_CANCELLED	Sau khi tài xế hủy chuyến (gửi đến tài xế)
/topic/notifications.user.{userId}	TRIP_CANCELLED_BY_DRIVER	Sau khi hủy chuyến (gửi đến từng hành khách bị ảnh hưởng)
/topic/notifications.user.{userId}	DRIVER_TRIP_COMPLETED	Sau khi hoàn thành chuyến (gửi đến tài xế)
/topic/notifications.user.{userId}	TRIP_COMPLETED	Sau khi hoàn thành chuyến (gửi đến từng hành khách)
/topic/notifications.user.{userId}	CASH_PAYMENT_CONFIRMED	Sau khi xác nhận tiền mặt (gửi đến hành khách)
/topic/notifications.user.{userId}	DRIVER_PROFILE_APPROVED	Admin duyệt hồ sơ (gửi đến tài xế – từ AdminDriverProfileService)

PHỤ LỤC II . PHẦN CODE – CÁC FILE LIÊN QUAN ĐẾN CÁ NHÂN THỰC HIỆN

Link GitHub: https://github.com/hungnx-yb/Ride_Up

2.1. Backend – Spring Boot (thư mục RideUp/)

Đường dẫn file	Chức năng
controller/ DriverTripController.java	Xử lý HTTP request cho toàn bộ API quản lý chuyến đi của tài xế: tạo chuyến, xem danh sách, xem chi tiết, bắt đầu/hoàn thành/hủy, xác nhận tiền mặt, thống kê
controller/ DriverProfileController.java	Xử lý HTTP request cho API hồ sơ tài xế: xem, cập nhật, nộp hồ sơ
controller/FileController. java	Xử lý API upload file ảnh lên Supabase Storage (CCCD, GPLX, ảnh xe)
service/DriverTripService. java	Toàn bộ nghiệp vụ quản lý chuyến đi: validate, tạo trip, quản lý trạng thái, xác nhận tiền mặt, thống kê, hoàn tiền tự động
service/ DriverProfileService.java	Toàn bộ nghiệp vụ quản lý hồ sơ tài xế: tạo/cập nhật/nộp hồ sơ, validate, quản lý Vehicle
service/FileService.java	Nghiệp vụ upload/delete/get URL file trên Supabase Storage
service/ NotificationRealtimePublisher.java	Phát sự kiện thông báo realtime qua WebSocket STOMP đến client
entity/Trip.java	Entity chuyến đi – ánh xạ bảng trip trong MySQL
entity/DriverProfile.java	Entity hồ sơ tài xế – ánh xạ bảng driver_profile
entity/Vehicle.java	Entity phương tiện – ánh xạ bảng vehicle
entity/Booking.java	Entity đặt chỗ – ánh xạ bảng booking (đọc để xử lý hủy chuyến, xác nhận tiền mặt)
entity/Payment.java	Entity thanh toán – ánh xạ bảng payment (cập nhật status khi xác nhận tiền mặt)
entity/TripPickupPoint.java	Entity điểm đón – ánh xạ bảng trip_pickup_point
entity/TripDropoffPoint.java	Entity điểm trả – ánh xạ bảng trip_dropoff_point

Đường dẫn file	Chức năng
repository/TripRepository.java	JPA repository với các query tùy chỉnh cho trip (JPQL, Lock, Aggregate)
repository/DriverProfileRepository.java	JPA repository cho driver_profile
repository/WardRepository.java	JPA repository với findByProvinceIdAndWardName() phục vụ resolve điểm đón/trả
dto/request/DriverTripRequest.java	DTO nhận request tạo chuyến đi từ client
dto/request/TripCancellationRequest.java	DTO nhận reason khi hủy chuyến
dto/request/DriverProfileUpdateRequest.java	DTO nhận dữ liệu cập nhật hồ sơ tài xế
dto/response/DriverTripResponse.java	DTO trả danh sách chuyến (tóm tắt)
dto/response/DriverTripDetailResponse.java	DTO trả chi tiết chuyến (bao gồm PointInfo và BookingInfo nested)
dto/response/DriverProfileResponse.java	DTO trả hồ sơ tài xế đầy đủ
enums/TripStatus.java	Enum trạng thái chuyến: OPEN, FULL, IN_PROGRESS, COMPLETED, CANCELLED
enums/DriverStatus.java	Enum trạng thái hồ sơ: PENDING, APPROVED, REJECTED
enums/BookingStatus.java	Enum trạng thái booking (đọc trong module Tài xế)
enums/PaymentMethod.java	Enum phương thức thanh toán: CASH, BANK_TRANSFER, VNPAY
enums/PaymentStatus.java	Enum trạng thái thanh toán: UNPAID, PAID, REFUNDED, FAILED
config/SupabaseStorageConfig.java	Cấu hình kết nối Supabase Storage (URL, API key, bucket name)

2.2. Mobile App – React Native (thư mục RideUpUI/)

Đường dẫn file / Component	Chức năng
screens/driver/ DriverHomeScreen.js	Trang chủ tài xế: thông kê tháng (số chuyến, doanh thu, rating), danh sách 5 chuyến gần nhất, FAB Tạo chuyến, skeleton loading, realtime refresh khi nhận WebSocket
screens/driver/ CreateTripScreen.js	Form tạo chuyến: ProvincePicker, WardPicker (chip), SeatStepper, giá vé, CalendarPicker, TimeSlot grid 48 slot, RevenuePreview, submit handler gọi POST /driver/trips
screens/driver/ AllTripsScreen.js	Danh sách chuyến với tab filter, TripCard có progress bar ghé, nút hành động trực tiếp trên card, Modal xác nhận hủy
screens/driver/ TripDetailScreen.js	Chi tiết chuyến: toàn bộ thông tin trip, điểm đón/trả, BookingCard list, nút xác nhận tiền mặt, nút Bắt đầu/Hoàn thành/Hủy
screens/driver/ DriverProfileScreen.js	Form hồ sơ tài xế: 3 nhóm thông tin, ImagePicker upload ảnh, nút Lưu và Nộp hồ sơ